

zTeX 宏集介绍

Eureka zongpingding5@outlook.com

由于本人时间有限, 目前此宏集的开发暂停.

September 21, 2025

1 简介

1.1 为何叫 $\LaTeX$?

为何宏集名称里有 ‘z’ 这个前缀, 这也许应是许多用户想知道的问题? 可能的原因:

- (1) 看到 $\LaTeX$ 3 开发团队用 “x” 来作为他们开发的一系列宏包前缀, 比如 `xparse`, `xcoffins`, `xfp` 等. 我便不能再使用 “x” 这一前缀了. 这个时候, 突然想到了一个字母 – “z”. 一方面 “ $x \rightarrow y \rightarrow z$ ”, 有了 “x”, 才有 “z” ($\LaTeX$ 全部基于 $\LaTeX$ 3 进行开发; 可以说, 没有 $\LaTeX$ 3, 就没有今天的 $\LaTeX$). 那么 “y” 去哪里了? 当作为用户的你 (you) 加入 $\LaTeX$ 使用者阵营后, 就有 “y” 了.
- (2) 你将 ‘z’ 逆时针旋转 90° , 就可以得到 “阿列夫 - $\aleph$ ”: 我希望 $\LaTeX$ 宏集能够有进一步 (无限) 拓展的可能; 这个宏集在设计之初, 便一直坚持可拓展性这一原则. 普通用户可以使用用户层面的命令, 模板制作者可以使用 $\LaTeX$ 提供的编程接口. 尽管 “ $\aleph\TeX$ ” 这个目标有些不切实际, 但是万一实现了呢?
- (3) 也许是看到了 $\text{\textcolor{brown}{TikZ}}$ 中的 “z”, 于是便以 ‘z’ 为本系列宏集的前缀了.

最开始的 $\LaTeX$ 宏集仅包含一个基本的 `zlatex.cls` 文档类, 而且原来的名称叫做 “ $\pi\LaTeX$ ”; 后面我又想基于 $\text{\textcolor{brown}{TikZ}}$ 开发一个绘图宏包, 用于实现常见平面图形的绘制以及外部程序的交互; 再后来发现 `beamer` 用起来很不方便, 便开发了 `slide` 库; 随着开发的不断深入, 我发现我已经在 `ztex.cls` 中写了很多十分有用的宏了, 于是我把这些宏分化了出来, 得到了 `ztool` 宏包, 得到了 `thm`, `cmd`, `font`, ... 这些模块, 以及 `slide`, `alias`, `thm` ... 这些库; 最终, $\LaTeX$ bundle 诞生了.

1.2 为何用 $\LaTeX$?

为什么要用我这个 $\LaTeX$ 宏集? $\text{\textcolor{brown}{TikZ}}$ 中负责和外部程序交互的那几个模块现在处于一种比较尴尬的境地, 用户如果会用这些程序, 那么你可以单独使用这些程序调整图片的所有细节, 最后在 $\LaTeX$ 中插入该图片. 如果用户不会使用这些外部拓展程序, 那么用户不仅需要先学习该程序的用法, 还需要学习 $\text{\textcolor{brown}{TikZ}}$ 宏集中对应命令的 $\LaTeX$ 语法; 这无疑是增加了用户的负担!

用户可以再思考这样一个问题: 我已经会用 $\LaTeX$ 自己写模板了, 为什么还要用别人的模板? 我如果不会用 $\LaTeX$ 写模板, 花费了大量的时间去了解一个庞大且复杂的模板的使用细节, 那么我为何不花费这些时间自己去学习 $\LaTeX$, 这样更能做出满足自己需求的模板? 最后还可以进一步推出: 我为什么一定要用 $\TeX$ 或 $\LaTeX$ 呢? 用 Word, Indesign 这些成熟的软件, 甚至是手写, 难道就不能写一篇规范的论文/笔记吗?

所以为什么 Knuth 老爷子要花费十年的时间去开发 $\TeX$ 呢?

上述的一系列推论正确吗? 仔细想一想, 上面的推导其实不都是正确的. 前一个条件并不一定是充分的, 或者说我们使用了一个假命题 (关系) 去得到了另一个命题 (关系).

根据基础的逻辑知识: 定义汇集 $R \vee S$ 为两关系 R, S 的逻辑析取, 定义汇集 $\neg R$ 为关系 R 的逻辑否定. 从而我们就可以定义所谓的“逻辑蕴含”关系 $\Rightarrow$, 即记号 $R \Rightarrow S$, 前者其实是如下的关系汇集:

$$S \vee (\neg R)$$

注记 1.1 其实有 $\neg, \vee$ 这两个基础的符号就已经能表示出很多的关系了; 比如逻辑合取记号: $R \wedge S$, 它其实就是: $\neg[(\neg R) \vee (\neg S)]$. 在规定逻辑公理后, 就可以用它们来说明常用的“三段论, 双重否定”等逻辑推理了. 比如我们常用的逆否命题就是说: 关系 $(R \Rightarrow S) \Rightarrow ((\neg S) \Rightarrow (\neg R))$ 是真的.

在我们定义了关系“真”后, 如果关系 $R \Rightarrow S$ 是真的, 那么:

- 当关系 R 为真的, 关系 S 必然是真的, 也就是我们得到了一个“真”的结论;
- 但如果 R, S 同时为假, 关系 $R \Rightarrow S$ 也是真的. 而此时我们的结论并不是“真的”, 也就是结论并不成立.

可以认为我们用一个假命题导出了另一个假命题, 下面说明 zTeX 值得你去用, 我将要如何去说服你呢?

让“ $R \Rightarrow S$ ”中的命题“ R ”为假就好了. zTeX 的上手难度相较于默认的 $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ 要低一点, 达到同样的排版效果, 你所花费的时间更少. 故上述“花费同样时间”这一个命题为假, 即“ zTeX 值得你用”这一命题成立. 你也许可以用其它的方式来反驳我, 但至少我找到了一个论据来说服我自己, 也找到了我开发这个宏集的初心.

1.3 项目维护

目前本项目已经在 GitHub, Gitlab, Gitee 上开源, 地址如下:

GitHub : https://github.com/zongpingding/zTeX_bundle

Gitlab : https://gitlab.com/zongpingding/zTeX_bundle

Gitee : https://gitee.com/zongpingding/zTeX_bundle

项目中包含: `ztex` 文档类, `zTiZ` 宏包, 以及 `ztool` 宏包的源码与用户手册. zTeX 宏集以 `lppl` 协议开源, 欢迎各位对源代码进行修改与二次分发. 若用户在使用此宏集的过程中发现任何的 Bug, 或想提出改进意见, 请在 Github 上提 Issue 或直接提交 PR.

请不要在 Gitee 或者是 Gitlab 上提问, 本人只维护 Github 上的仓库; 尽管有时可能会为了国内用户下载方便, 把 Github 仓库中的内容同步到这两处. 后续的开发过程中, 三者不会同步更新, 请以 Github 仓库为准.

本项目为完全免费、纯属兴趣驱动 (为爱发电) 之作. 对于任何使用本模板所引发的严重后果, 我概不负责. 我非常乐意帮助大家解决问题, 但在提问之前, 请务必先了解 $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ 的提问规范.

当前宏集的稳定版本于半年之前发布, 最新的开发版请切换到 “dev” 分支; 本手册适用于当前最新的开发版. 请到: [Release 界面](#) 下载.

1.4 基本组成

$\LaTeX$ 宏集包含如下内容:

- `ztex` 文档类;
- `ztikz` 宏包;
- `ztool` 宏包;
- `zslide` 宏包 (不推荐使用).

$\LaTeX$ 宏集独立实现了一个 `ztool` 宏包, 它是 $\LaTeX$ 宏集中各文档类或宏包的基础. 此宏包中包含原来已被废弃的 `l3sys-shell` 中的所有命令. 除此之外, `ztool` 提供了 `box` 操作, 文件 IO 以及基本图形绘制相关的函数. 在 `ztool` 的协助下, $\LaTeX$ 能够避免或减少命令行 `-shell-escape` 参数或其它相关宏包的调用 (如 `robust-externalize` 宏包).

`ztex` 文档类对标 `memoir`, `koma-script` 宏集, 用于生成书籍或演示文稿. 尽管在 $\LaTeX$ 中, 直接将 `layout/slide` 选项置为 `true` 即可生成演示文档, 但该库目前很不成熟, 在严肃场合中, 推荐使用原始的 `beamer` 或 `ctexbeamer` 文档类.

`ztikz` 宏包提供了绘制平面图形以及调用外部程序的接口¹. `zslide` 宏包是自己临时设计的一套 `beamer` 主题, 还未进行常规测试, 请谨慎使用.

从本介绍文档即可看出, 本模板整体风格较为朴素, 未采用华丽的配色方案或精致的页面设计. 然而, 在长时间尝试和调试 $\LaTeX$ 模板的过程中, 我逐渐发现这种简洁质朴的风格最符合广大 $\LaTeX$ 用户的使用习惯与审美偏好. 若你更倾向于精美的排版风格, 亦可参考其他的模板, 如 `Elegant $\LaTeX$` 、`Beauty $\LaTeX$` 等.

1.5 用户手册

普通 $\LaTeX$ 用户可跳过本文档的“节 (3)”. 该部分主要记录了我对本模板设计思路的说明, 以及个人在编写 $\LaTeX$ 过程中的一些体会, 对模板或宏包的实际使用并无直接帮助. 若你希望了解 `ztex` 文档类的具体用法, 请参阅 `zlatex_interface.pdf`; 若需了解 `ztikz` 宏包的使用方法, 请参阅 `ztikz_interface.pdf`. 目前 `zslide` 宏包尚无详细文档, 仅提供了示例文件 `zslide_manual.pdf` 供用户参考. `ztool` 宏包主要为模板的开发者准备, 普通用户无需阅读.

¹众所周知, 在 $\LaTeX$ 中绘图是一件十分痛苦的事, 于是乎你会看到很多书籍或笔记中的图形都是手绘或截图, 并非矢量图

2 安装使用

2.1 在线模板

为了让部分用户可以直接使用到 $\text{\texttt{zT}}_{\text{E}}\text{X}$, 免去“繁杂”的环境配置. 我已将本模板部署在 $\text{\texttt{T}}_{\text{E}}\text{X}$ Page 上, 地址为: [\text{\texttt{TeX}}\text{Page} \text{\texttt{zT}}_{\text{E}}\text{X} \text{Project}](#), 直接打开此地址即可体验. 由于技术原因, $\text{\texttt{zT}}_{\text{E}}\text{X}$ 请在本地体验.

2.2 本地安装

$\text{\texttt{zT}}_{\text{E}}\text{X}$ 宏集目前还未上传 CTAN, 因为还没有开发完成. 本文档类使用的部分 $\text{\texttt{L}}_{\text{A}}\text{\texttt{T}}_{\text{E}}\text{\texttt{X}}3$ 命令在老版本的 $\text{\texttt{T}}_{\text{E}}\text{\texttt{X}}\text{Live}$ 下并不存在, 若用户的 $\text{\texttt{T}}_{\text{E}}\text{\texttt{X}}\text{Live}$ 版本过低, 则可能无法正常使用本宏集. 目前 $\text{\texttt{zT}}_{\text{E}}\text{X}$ 文档类在各平台的兼容情况为:

Windows : $\text{\texttt{T}}_{\text{E}}\text{\texttt{X}}\text{Live}$ 最低版本 2024

Linux : $\text{\texttt{T}}_{\text{E}}\text{\texttt{X}}\text{Live}$ 最低版本 2024

MacOS : $\text{\texttt{MacT}}_{\text{E}}\text{\texttt{X}}$ 还未测试

因 $\text{\texttt{zT}}_{\text{E}}\text{X}$ 还未传入 CTAN(未来可能会考虑), 所以想要使用此文档类, 只有如下两种方法:

- 把此宏集 - $\text{\texttt{ztex}}$ 目录中的所有内容放入当前项目文件夹下;
- 在命令行运行命令: $\text{\texttt{kpsewhich -var-value=TEXMFHOME}}$, 在 Windows 上这个路径一般是: $\text{C:}/\text{Users}/\langle\text{name}\rangle/\text{texmf}/$, 在 Linux 下一般是: $\sim/\text{texmf}/$; 具体路径以自己的实际情况为准. 在此路径下新建文件夹 $\text{\texttt{tex/latex/ztex}}$; 此文件夹对应的路径我们记为 $\langle\text{\texttt{zT}}_{\text{E}}\text{\texttt{X}}\rangle$, 随后把 $\text{\texttt{ztex}}$ 目录中的所有内容放入 $\langle\text{\texttt{zT}}_{\text{E}}\text{\texttt{X}}\rangle$ 下即可.

在本手册后续, 我们使用 $\langle\text{\texttt{zT}}_{\text{E}}\text{\texttt{X}}\rangle$ 表示本宏集的根本目录.

NOTE: 如果用户不需要使用 alias 库, 那么一些比较老 $\text{\texttt{T}}_{\text{E}}\text{\texttt{X}}\text{Live}$ 也能运行此宏集.

3 开发过程

本模板的设计经历了较长时间的积累与迭代。最初接触 $\text{\LaTeX}$ 时，我只是将常用的宏整理进一个 `.sty` 文件中，误以为这便是一个宏包（实际上它称得上是一个宏包）。随后接触到了 **Elegant $\text{\LaTeX}$** 系列模板，并曾使用其中的 `elegantbook` 文档类撰写笔记。然而，随着使用深入，我逐渐发现模板默认的样式并不完全符合个人需求，许多细节希望能够自行定制。遗憾的是，当时对 $\text{\LaTeX}$ 的理解尚浅，面对复杂的模板源码无从下手（打开任何一个模板，映入眼帘的源码对于我来说与一堆乱码无异）。后续通过查阅资料、阅读相关文章，逐步积累经验，渐渐熟悉了 $\text{\LaTeX}$ 中的各种命令与机制，才最终开始着手本模板的独立设计。

$\text{\LaTeX}$ 的第一版基本是在 `elegantbook` 文档类的基础上修改而成，仅在字体、配色等方面做了一些简单调整。然而，随着功能的不断叠加，模板逐渐变得混乱，代码结构也变得难以维护²。其中，键值对接口的实现对于我来说尤为困难。以文档类语言切换功能为例，当时通过 `\ifdefstring` 实现，以下是当初的相关代码片段：

```
1 \DeclareVoidOption{cn}{\kvs{lang=cn}} 1
2 \DeclareVoidOption{en}{\kvs{lang=en}} 2
3 \DeclareStringOption{cn}{lang} 3
```

代码的书写过程颇为繁琐。当时模板仍以 `article` 文档类为基础，缺乏许多 `book` 文档类中内置的计数器与章节结构，不得不自行声明相关命令。然而，自定义的命令常与其他宏包不兼容，尤其是在集成 `hyperref` 宏包时问题频出。由于计数器定义不规范，导致跳转功能异常。例如，使用 `\label` 时，所激活的跳转目标往往并非正确的章节位置，目录中的链接也存在类似问题，使用体验大打折扣。

另一方面，初代 $\text{\LaTeX}$ 文档类完全基于 $\text{\LaTeX}2\epsilon$ 构建，许多宏展开相关的代码写的不仅繁琐，逻辑也很混乱。当时经验有限，模板中的大多数解决方案都借鉴（抄袭）自 **[TeX-StackExchange](#)** 上的回答，导致整个模板虽然“能跑”，但对其中许多命令的具体作用并不真正理解，并不清楚这些“解决方法”会不会产生一些不为人知的副作用。

²事实上，最初 `ztex` 与 `ztikz` 宏包是写在一起的，整体结构非常凌乱。

3.1 ztex

后来, 我将 ztikz 宏包从原有的 ztex 文档类中剥离出来, 并使用 L^AT_EX3 对原始文档类和 ztikz 进行了重构。z_LT_EX 文档类默认基于 article 文档类构建, 同时也支持加载其他文档类。此阶段的开发理念发生了显著变化: 在添加任何的配置前, 我都会事先明确其提供的功能, 了解该配置需要的依赖, 这一配置对已有的代码或宏包有无影响, ..., 然后再自行编写代码实现。由此, z_LT_EX 的开发正式开始了。事实证明, 基于 L^AT_EX3 的重构极大提升了代码的清晰度和整体开发效率。以下为当时 ztex 文档类选项的相关声明:

```

4 \zlatex_define_option:n {                                4
5   % language                                              5
6   lang .str_gset:N = \g__zlatex_lang_str,                6
7   lang .initial:n = { en },                              7
8   % page layout                                          8
9   layout .str_gset:N = \g__zlatex_layout_str,            9
10  layout .initial:n = { twoside },                        10
11  % margin option                                       11
12  margin .bool_gset:N = \g__zlatex_margin_bool,          12
13  margin .initial:n = { true },                          13
14 }                                                       14
15 \ProcessKeysOptions {zlatex / option}                   15

```

看起来确实清爽了许多, 但很快我意识到, 这样的实现方式在实际使用中仍不够灵活。问题在于: 当需传递给子文档类的选项较多时, 必须逐一声明大量键值对; 而当整个文档类中键值对数量庞大时, 维护成本显著增加。为了解决这一问题, 我引入了 l3keys 提供的元键机制 (`.meta:nn`)。其核心作用在于: 通过模块化管理各类键值对, 实现层级式组织与调用, 从而提升代码的可读性与扩展性。以下是当时 ztex 文档类中键值接口的实现代码:

```

16 \zlatex_define_option:n {                                16
17   % zlatex language                                      17
18   lang .str_gset:N = \g__zlatex_lang_str,                18
19   lang .initial:n = { en },                              19
20   % class and options                                    20
21   class .str_gset:N = \g__zlatex_subclass_type_str,      21
22   class .initial:n = { book },                           22
23   classOption .clist_gset:N = \g__zlatex_subclass_option_clist, 23
24   classOption .initial:n = { onside, 10pt },             24
25   % zlatex options meta key                             25
26   layout .meta:nn = {zlatex / layout}{#1},              26
27   mathSpec .meta:nn = {zlatex / mathSpec}{#1},          27
28   font .meta:nn = {zlatex / font}{#1},                  28
29 }                                                         29

```

为了轻松处理子文档类选项的加载问题, 我引入了 `<classOption>` 这个键。

3.2 ztikz

开发宏包 ztikz 也花了我很多的时间, ztikz 从最开始的一个小宏包变成了一个拥有众多拓展库的庞然大物. 这段时间, 我为 ztikz 宏包开发了 cache, python, gnuplot, wolfram 和 l3draw 库. 这些库可以先通过下面的命令进行声明:

```
30 \ProvidesExplFile{ztikzmodule.cache.tex}{2024/06/15} 30
31 {1.0.0}{cache~module~for~ztikz} 31
```

然后在主宏包 ztikz 中使用如下命令进行调用:

```
32 \cs_new_nopar:Npn \g__ztikz_load_module:n #1 32
33 { 33
34   \clist_map_inline:nn {#1} 34
35   { \file_if_exist_input:nF {modules/ztikzmodule.##1.tex}{ } } 35
36 } 36
37 \NewDocumentCommand\ztikzLoadModule{m} 37
38 { 38
39   \g__ztikz_load_module:n {#1} 39
40 } 40
```

划分出 ztikz 的库后, 宏包使用者只需通过如下的命令就可以轻松调用:

```
41 \ztikzLoadModule{cache, python} 41
```

而且, 将一个宏包划分为一个个的库来开发这一行为, 不仅可以方便宏包的使用者, 更让宏包的开发者可以聚焦于单个库的开发, 这极大地提高了我的开发效率.

在开发 ztikz 的 cache 库时, 我遇到了数不清的困难, 包括但不限于:

- 怎么将一个环境中的内容不加改变地输出到外部文件中?
- 怎么为每一个需要缓存的内容“打”上一个唯一的“身份标签”?
- 为什么同样都是字符串, 但是 string 和 token list 在 \tl_if_eq:nn 中就是判断为不相等?
- 怎么调用上一次的缓存结果?
- 怎么临时忽略缓存机制, 或强制调用上一次的缓存结果?
- 怎么提供对应的编程接口?
- ...

虽然, 上述的问题目前均已解决, 但目前的 cache 库仍有缺陷:

- 无法去除 tikz 的 externalize 库依赖, 我自己还没有能力自己写一个 externalize 库出来.

- 无法提供与 Matlab 的交互接口.
- cache 库提供的普通用户接口仍然过于复杂.
- ...

3.3 ztool

大概是开发到中后期的时候, 我发现我在 `ztex` 或 `ztikz` 中定义了大量与此宏包无关的宏, 比如 “`TEX` 盒子操作”, “`shell-escape`”, “文件 IO 操作”; 然后我便把这些宏分离到了 `ztool` 宏包中. 上面的这些功能几乎时没有什么关联的, 后面我更是在 `ztool` 宏包内将它们划分为了下面的这几个部分:

- `shell-escape`,
- `file-io`,
- `box`,
- `zdraw`;

它们之间互不干扰, 用户在使用时仅需加载其需要的部分即可; 比如用户需要使用 `file-io` 中的一个宏, 他只需要使用如下的命令:

```
42 \ztoolloadlib{file-io}
```

42

此时, `ztool` 仅会加载 `file-io` 相关的宏, 其它部分的宏则不会被加载. `ztool` 实现这一机制同样是使用了上述方法 – 将 `ztool` 划分为一个个的库.

3.4 l3build

我之前完全没有接触过 L^AT_EX 相关的“代码测试”内容, 一个偶然的时间, 我发现了 l3build. 我们写的代码是需要测试的: 你需要确保后续开发的代码不会影响之前的代码, 怎么保证呢? 写好单元测试, 每次添加新功能后就跑一跑单元测试, 如果全部的测试都通过了, 那么你后续的开发是没问题的. 当然, 你的单元测试必须得写全面了.

最开始的自己很懒, 不想写测试, 觉得费时间, 多写一点代码不好吗? 但若你后续写的代码破坏了前面已有的功能, 这段代码就是没有意义的. 所以要勤于写单元测试!

4 宏集设计

4.1 设计参考

本系列自诞生以来始终由我个人独立开发，过程中借鉴了诸多优秀的文档类与宏包。其中，参考最多的是 `CTEXart` 文档类，它为本项目提供了主要的设计思路，该文档类完全基于 `LATEX3` 编写，在选项配置模块方面，它给了我很多启发。

`zlTEX` 宏集中的文档类或宏包的 Key-Value 接口先是参考了 `TEX-StackExchange` 上的相关讨论，然后再采用了 `LATEX3` 的 `l3keys` 模块实现。此方案的优点是显而易见的：配置接口简洁明了、符合用户习惯、同时也便于模板的后续维护与扩展。

在后续的开发过程中，`CUSTeX` 宏集也为我带来了诸多启发，我参考了其中许多优秀的设计方案。尤其值得一提的是该项目将“用户接口”与“编程接口”进行区分的思想，对此宏集后续的开发影响颇深。

4.2 设计原则

说实话，这个标题可能有些夸大了——“设计原则”究竟指的是什么，我自己也不清楚。我只是希望我的模板看起来足够舒服而已。那怎样才能让一个模板“看着舒服”呢？我也无法给出明确答案。但至少，它应该与页边距、字体大小、字体样式等因素有关系。更进一步地说，这些因素并非彼此独立，而是相互制约、共同作用的。举例而言，当页边距增大、版心变小时，正文字体的大小也应随之调整，以维持整体的视觉平衡和可读性。

当时遇到了一个问题：一行设置多少个字符才合适？在查阅 TeX StackExchange 相关讨论后发现，对于英文文本来说，一行包含 65–90 个字母被认为是较为理想的范围，且常见的正文字体尺寸为 10pt、11pt 或 12pt。

至于页边距应如何设置，我参考了 `elegantbook`、`ctextart` 等文档类的设计，也逐渐总结出一些经验。起初，测量页面布局中的各项距离是非常不方便的，我都是动用尺子手动测量的。后来我发现了一个非常实用的宏包——`fgruler`，它可以在生成的 PDF 中直接显示页面布局的尺寸信息，且使用方法也非常简便：

43 `\usepackage[hshift=0mm,vshift=0mm]{fgruler}`

43

当你在导言区加入上述配置后，生成 PDF 的每页都能看到如图 (1) 这样的输出。我终于摆脱使用尺子手动量这一方法了！

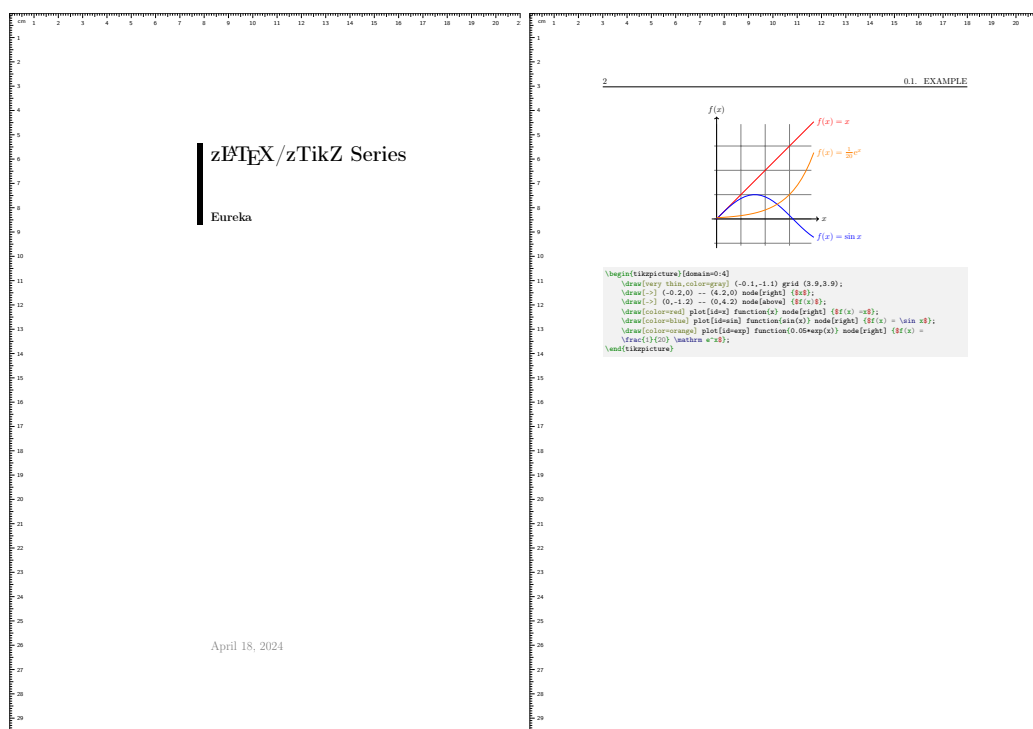


图 1: 页面布局示意图

在设计本宏集时，我始终在字体配置上有所犹豫：是否应将字体打包进模板？是否应

在模板中为用户设置默认字体？在本宏集的最初版本中，我尝试收集了一些免费的中英文字体，并直接放置在模板的文件夹中。然而，这种做法也带来了不少问题：

- 部分用户真的需要该字体吗？增加的字体会变成模板或用户的负担吗？
- 该字体可以随意传播吗？万一某个用户将该字体进行了商用？
- 部分中文字体包含的字形往往是不全的，怎么解决？
- ...

最终的处理办法：本宏集不打包任何的字体，但添加部分 $\text{T}_{\text{E}}\text{X Live}$ 内置字体配置；宏集本身提供字体设置的接口，但所有的字体定义与样式由用户指定。除此之外， $\text{\LaTeX}$ 还提供了数学字体配置接口，以供用户选用。

在开发 $\text{\LaTeX}$ 宏集的过程中，行距等排版细节也曾让我困扰许久。实际上，设计一个模板需要考虑的因素远比预期复杂，几乎每一个参数的设置都会相互影响。不过，在反复尝试与调整的过程中，我也逐渐总结出—条经验：对于一时把握不准的配置，就保留默认设置。

Be simple, be fool – 保持简单，反而更容易达到稳定和谐的效果。

尽管在开发过程中遇到了诸多困难， $\text{\LaTeX}$ 最终仍未烂尾，顺利完成并呈现在了大家面前。

4.3 无题

时至今日, 再次回头来看我的这个模板, 我反而有了一些其他的感受. 一个模板到底需要给用户定制什么东西? 到底需要给用户多大的自由空间 (配置选项)? 如果你的配置选项过多, 像 `koma-script`, `Memoir` 那样, 模板作者给用户处理了很多的细节, 提供了种类繁多的接口. 或者像部分简单的模板仅提供几个必要的设置和命令; 而且, 如果一个模板的说明文档都达到了上百页, 那么我作为一个用户为什么不自己学习做模板, 写一个适合自己的模板, 反而要话这部分时间来学习使用你的模板? 如果模板的配置选项过少, 那么用户又会觉得这个模板不够灵活. 所以, 到底什么样的一个模板设计才能够称得上是: **简单, 灵活, 易用**? 遗憾的是, 现在我也没有办法回答这个问题, 所以这个问题作为习题, 留给使用者回答了...

发展至今, $\LaTeX$ 宏集早已不再是一个简单的“文档类 + 绘图库 + 幻灯片”集合, 这也使得它并不适合 $\LaTeX$ 初学者使用. 在开发的过程中, 我也逐渐意识到: 很多时候, 我们并不一定需要亲自设计一个模板. 更合理的做法或许是 – 根据自己的需求, 选择合适的功能性宏包, 并通过它们提供的接口实现所需的功能. 这种方式不仅更贴合实际使用场景, 也减少了与其他宏包的兼容性问题, 更无需投入大量时间去理解第三方模板的结构与细节.

实际上, `article`、`book` 等基础文档类, 加上丰富的功能宏包, 已经足以满足绝大多数排版需求. 也许我们并不需要再去重复造一个模板的“轮子”. 相比之下, 我更认同将精力投入到基础性宏包的开发上, 就如 `pgf`、`l3draw` 等优秀项目所做的那样 – 它们专注于提供一组底层的绘图或功能接口, 将更高层的封装留给用户根据自身需求自行实现.

Happy $\LaTeX$ ing !

>_<

5 文档指南

5.1 记号说明

本宏集的所有用户手册均遵守如下规范:

- 命令和键值对采用打字机字体;
- 键的默认值通过加粗标明, 并且与右侧蓝色文本一致;
- 所有命令排版格式为: `\cmd[oArg]{pArg}`;
- 所有键值排版格式为: `<key> = value`;

5.2 复制样例

L^AT_EX 宏集的所有用户手册均提供了大量示例及其对应的代码。为提升阅读体验, 在排版过程中对部分代码抄录环境中的符号进行了格式上的调整。例如:

- 在示例代码中, 换行符可能以“↵”表示, 复制代码时请将该符号删除;
- 若示例中包含行号, 请在复制后手动去除多余的行号;
- 此外, 在后续的 Implementation 节中, 部分代码因排版原因进行了换行, 使用时请根据实际情况去除不必要的换行符, 以确保代码能够正确编译。

5.3 键值指定

本系列中的大多数命令均采用键值对形式调用，因此，如果某个命令的可用键较多，而用户手册中的说明又较为模糊，用户可参考手册末尾 Implementation 部分中该命令的声明原型。该部分列出了该命令所支持的所有键及其默认值，有助于进一步理解和正确使用命令。下面以具体命令 `\Polygon` 为例，说明如何使用键值对接口：

```

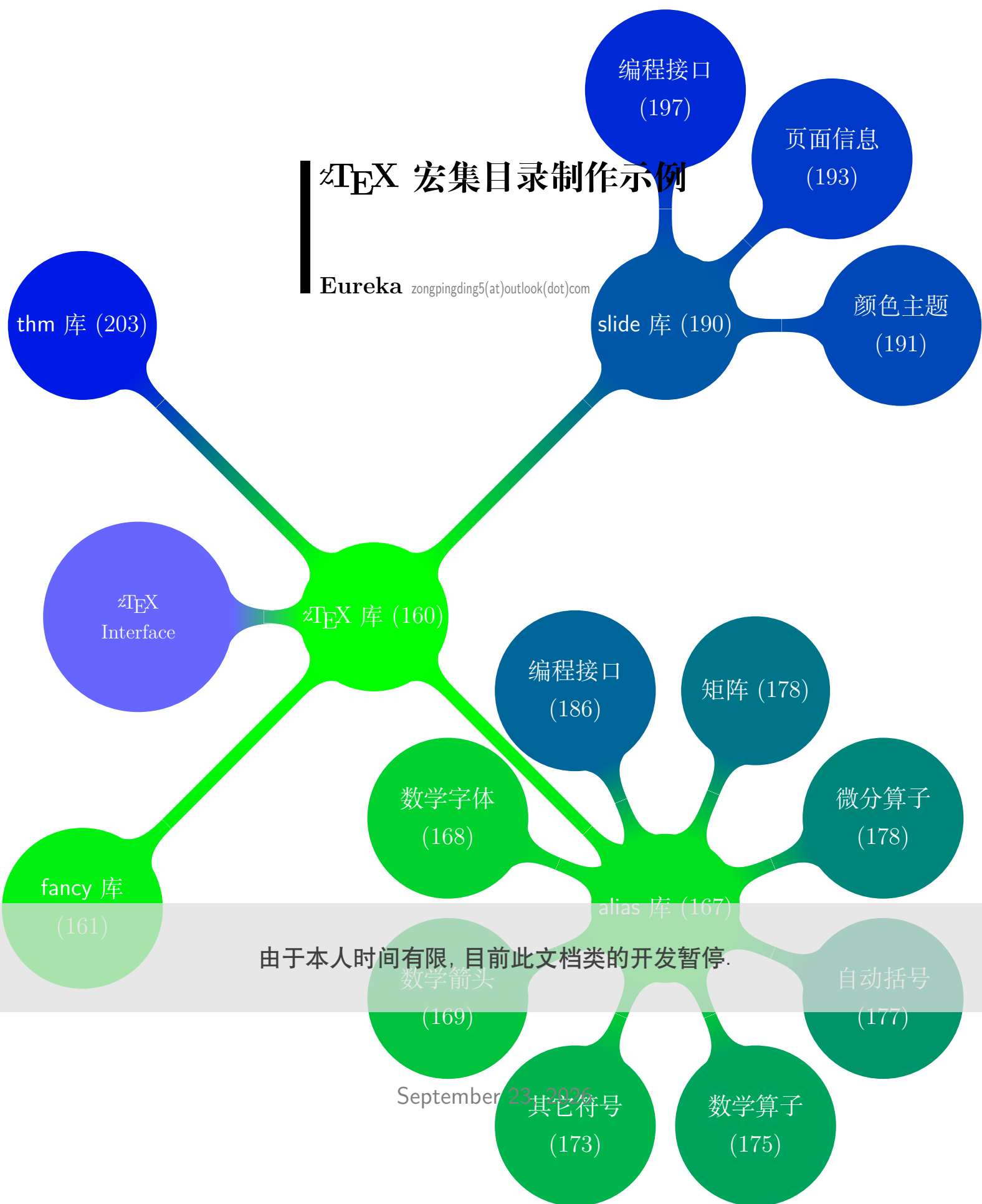
44 % key-value setup                                     44
45 \keys_define:nn { ztikz / polygon }                 45
46 {                                                     46
47     radius      .fp_set:N = \l__polygon_radius_fp,   47
48     radius      .initial:n = { 1 },                 48
49     edgeColor    .tl_set:N = \l__polygon_edge_color_tl, 49
50     edgeColor    .initial:n = { black },             50
51     fillColor    .tl_set:N = \l__polygon_fill_color_tl, 51
52     fillColor    .initial:n = { white },             52
53     fillOpacity  .fp_set:N = \l__polygon_fill_opacity_fp, 53
54     fillOpacity  .initial:n = { 0 },                 54
55     rotate       .fp_set:N = \l__polygon_rotate_angle, 55
56     rotate       .initial:n = { 0 },                 56
57     shift        .tl_set:N = \l__polygon_shift_tl,    57
58     shift        .initial:n = { (0,0) },             58
59     marker       .tl_set:N = \l__polygon_marker_option_tl, 59
60     marker       .initial:n = { },                  60
61 }                                                     61
62 % command                                             62
63 \NewDocumentCommand\Polygon{ 0{}m }                 63
64 {                                                     64
65     \group_begin:                                     65
66     \keys_set:nn { ztikz / polygon } { #1 }          66
67     ...                                               67
68     \group_end:                                       68
69 }                                                     69

```

上述 `\Polygon` 命令解读：第一个参数为可选参数 (0 类型)，通过键值对进行指定。可用的键有：`<radius>`、`<edgeColor>`、`<fillColor>`、`<fillOpacity>`、`<rotate>`、`<shift>`、`<marker>` 等。键 `<radius>` 接受一个浮点数 (参考后面的“`\fp_set:N`”), 默认值为 1 (参考后面的“`.initial:n = { 1 }`”); 再比如, 键 `<edgeColor>` 可接受一个 tokenlist (参考后面的“`\tl_set:N`”), 默认值为 “black” (参考后面的“`.initial:n = { black }`”).

zT_EX 宏集目录制作示例

Eureka zongpingding5(at)outlook(dot)com



总目录

1 基本介绍	1	6.7.2 token 命令	77
2 安装使用	2	6.8 sect 模块	83
2.1 在线模板	2	6.8.1 序言	83
2.2 本地安装	2	6.8.2 层级/模板	84
2.3 快速开始	3	6.8.3 章节标题	85
3 基本命令	5	6.8.4 章节目录	91
4 文档类选项	7	6.8.5 使用案例	105
5 状态检测	12	6.8.6 编程接口: 初始化	131
6 $\LaTeX$ 模块	13	6.8.7 编程接口: 章节命令	134
6.1 font 模块	14	6.8.8 编程接口: 目录	137
6.1.1 字体机制	14	6.8.9 编程接口: 模板	150
6.1.2 默认字体族	17	6.8.10 编程接口: 杂项	152
6.1.3 新建字体族	17	6.9 sclist 模块	154
6.1.4 切换字体	19	6.10 graphics 模块	158
6.1.5 $\LaTeX$ 接口	20	6.11 counter 模块	159
6.1.6 杂项	24	7 $\LaTeX$ 库	160
6.2 ref 模块	25	7.1 fancy 库	161
6.2.1 超链接	25	7.3 alias 库	167
6.2.2 标签与引用	28	7.3.1 数学字体	168
6.2.3 杂项	30	7.3.2 数学箭头	169
6.3 page 模块	31	7.3.3 其它符号	173
6.3.1 页面布局	31	7.3.4 数学算子	175
6.3.2 页眉页脚	32	7.3.5 自动括号	177
6.3.3 页面水印	35	7.3.6 微分算子	178
6.3.4 杂项	37	7.3.7 矩阵	178
6.4 color 模块	38	7.3.8 编程接口	186
6.5 thm 模块	41	7.4 slide 库	190
6.5.1 用户接口	42	7.4.1 颜色主题	191
6.5.2 定理目录	48	7.4.2 页面信息	193
6.5.3 高级接口	51	7.4.3 编程接口	197
6.5.4 环境钩子	55	7.5 thm 库	203
6.6 box 模块	59	8 ztool 宏包	211
6.7 cmd 模块	69	9 TODO	212
6.7.1 列表补丁	75	10 $\LaTeX$ 源码	219
		10.1 ztex.cls	220

10.2	ztex.options.tex	231	10.3.11	counter	376
10.3	Module	236	10.3.12	graphics	377
10.3.1	box	236	10.4	Library	379
10.3.2	font	244	10.4.1	fancy	379
10.3.3	ref	249	10.4.2	cmd	382
10.3.4	page	255	10.4.3	alias	386
10.3.5	color	264	10.4.4	slide	401
10.3.6	thm	268	10.4.5	thm	417
10.3.7	sect	287	10.4.6	primitive	425
10.3.8	sclist	353	10.4.7	pratt	426
10.3.9	cmd	361			
10.3.10	item	375	11	索引	427

1 序言

最后附上一些复杂的目录格式定制示例，涵盖目录样式设置以及目录数据提取，可作为用户深度定制的参考。下面这段代码展示了后续样例会用到的一些命令和依赖宏包：

```

1 % \usepackage{tikz}
2 % \usepackage{tikz-qtrees}
3 % \usepackage[log]{tikz-qtrees-patch}
4 % \usetikzlibrary { mindmap, trees }
5 \definecolor{SeaGreen}{rgb}{0.18, 0.55, 0.34}
6 \def\customtothemecolor{SeaGreen}
7 \ExplSyntaxOn\makeatletter
8 \dim_new:N \linewidthfix
9 \def\tocupdatelinenwidthfix{
10   \dim_set:Nn \linewidthfix {\linewidth-2\fbboxsep}
11 }
12 \def\ornamentcorner#1#2#3{
13   \node[anchor=#1, rotate=#3, outer~sep=0pt, inner~sep=0pt] at (frame.#2)
14   {\pgfornament[width=.8cm, color=\customtothemecolor]{10}};
15 \def\ztochyperpage#1{\exp_args:Nne \hyper@link{link}{page.#1}{#1}}
16 \makeatother\ExplSyntaxOff

```

2 测试数据

本手册在排版目录时用到的测试数据：“data.toc”，“data_II.toc”，“data.ptoc”。data.ptoc 文件仅用于向读者展示 sect 模块中 keyval seq 的存储结构；当“dump-ptable = true”时， $\LaTeX$ 会自动生成该文件。

<pre> \contentsline {section}{1}{AAA-1}{1}{} \contentsline {subsection}{1.1}{BBB-1}{1}{} \contentsline {subsubsection}{1.1.1}{CCC-1}{1}{} \contentsline {subsubsection}{1.1.2}{CCC-2}{1}{} \contentsline {subsubsection}{1.1.3}{CCC-3}{1}{} \contentsline {subsubsection}{1.1.4}{CCC-4}{1}{} \contentsline {subsubsection}{1.1.5}{CCC-5}{1}{} </pre>	data.toc
--	----------

```

\contentsline {subsection}{1.2}{BBB-2}{1}{}
\contentsline {subsubsection}{1.2.1}{CCC-6}{1}{}
\contentsline {subsubsection}{1.2.2}{CCC-7}{1}{}
\contentsline {subsubsection}{1.2.3}{CCC-8}{1}{}
\contentsline {subsubsection}{1.2.4}{CCC-9}{1}{}
\contentsline {subsection}{1.3}{BBB-3}{1}{}

```

```

\contentsline{section}{1}{AAA-1}{1}{}% data_II.toc
\contentsline{subsection}{1}{BBB-1}{1}{}%
\contentsline{subsubsection}{1.1.1}{CCC-1}{1}{}%
\contentsline{subsubsection}{1.1.2}{CCC-2}{2}{}%
\contentsline{subsubsection}{1.1.3}{CCxxxC-3}{3}{}%
\contentsline{subsubsection}{1.1.4}{CCssgda\_shgfeC-4}{100}{}%
\contentsline{subsubsection}{1.1.5}{CsasfCC-5}{123}{}%
\contentsline{subsubsection}{1.1.6}{CasklklCC-x}{999}{}%
\contentsline{subsection}{2}{B BB-2}{1}{}%
\contentsline{subsubsection}{1.2.1}{CCC-6}{1}{}%

```

```

class={section},name=1,title={AAA-1},page=1,anchor={section.1},raw=1,ptoc
\contentsline {section}{1}{AAA-1}{1}{}
class={subsection},name=1.1,title={BBB-1},page=1,anchor={subsection.1.1},
raw={\contentsline {subsection}{1.1}{BBB-1}{1}{}
class={subsubsection},name=1.1.1,title={CCC-1},page=1,
anchor={subsubsection.1.1.1},raw={\contentsline
{subsubsection}{1.1.1}{CCC-1}{1}{}
class={subsubsection},name=1.1.2,title={CCC-2},page=1,
anchor={subsubsection.1.1.2},raw={\contentsline
{subsubsection}{1.1.2}{CCC-2}{1}{}
class={subsubsection},name=1.1.3,title={CCC-3},page=1,
anchor={subsubsection.1.1.3},raw={\contentsline
{subsubsection}{1.1.3}{CCC-3}{1}{}
class={subsubsection},name=1.1.4,title={CCC-4},page=1,
anchor={subsubsection.1.1.4},raw={\contentsline
{subsubsection}{1.1.4}{CCC-4}{1}{}

```

```

class={subsubsection},name={1.1.5},title={CCC-5},page={1}, ✓
anchor={subsubsection.1.1.5},raw={\contentsline ✓
{subsubsection}{1.1.5}{CCC-5}{1}{}}
class={subsection},name={1.2},title={BBB-2},page={1},anchor={subsection.1.2}, ✓
raw={\contentsline {subsection}{1.2}{BBB-2}{1}{}}
class={subsubsection},name={1.2.1},title={CCC-6},page={1}, ✓
anchor={subsubsection.1.2.1},raw={\contentsline ✓
{subsubsection}{1.2.1}{CCC-6}{1}{}}
class={subsubsection},name={1.2.2},title={CCC-7},page={1}, ✓
anchor={subsubsection.1.2.2},raw={\contentsline ✓
{subsubsection}{1.2.2}{CCC-7}{1}{}}
class={subsubsection},name={1.2.3},title={CCC-8},page={1}, ✓
anchor={subsubsection.1.2.3},raw={\contentsline ✓
{subsubsection}{1.2.3}{CCC-8}{1}{}}
class={subsubsection},name={1.2.4},title={CCC-9},page={1}, ✓
anchor={subsubsection.1.2.4},raw={\contentsline ✓
{subsubsection}{1.2.4}{CCC-9}{1}{}}
class={subsection},name={1.3},title={BBB-3},page={1},anchor={subsection.1.3}, ✓
raw={\contentsline {subsection}{1.3}{BBB-3}{1}{}}

```

3 案例:1

一个很简单的案例:

例 1

```

\begin{group}
\zotocformat\subsection
{
  format+=\color{teal},
  leader.sep=1pt,
  leader.raise=2.5pt,
  page.width=10pt
}
\zotocgroupinsert{subsection,1,begin}%
{%
  \begin{framed}%
  \pgfornament[width = 2cm,color = teal]{67}%
  \quad\rule[-5em]{.5pt}{10em}%
  \begin{minipage}{.75\linewidth}%
  }
\zotocgroupinsert{subsection,1,end}%
{%
  \end{minipage}%
  \end{framed}%
}
\zlocaltoc{subsection}{4}
\end{group}

```

6.1 font 模块.....	14
------------------	----



6.1.1	字体机制	14
6.1.2	默认字体族	17
6.1.3	新建字体族	17
6.1.4	切换字体	19
6.1.5	LaTeX 接口	20
6.1.6	杂项	24

4 案例:2

下面这个案例清晰地展示了目录中这些 hook 的位置关系, `BG:<int>` 和 `EG:<int>` 代表目录条目自身局部组的开始和结束位置 (这些局部组目前已被移除 – 2025-09-12).

```
\ExplSyntaxOn
\int_new:N \g__test_toc_group_int
\AddToHook{ztoc/tocline/begin}
{
  \fbox{BG:\int_use:N \g__test_toc_group_int}
}
\AddToHook{ztoc/tocline/end}
{
  \fbox{EG:\int_use:N \g__test_toc_group_int}
  \int_gincr:N \g__test_toc_group_int
}
\cs_new:Npn \__debug_ztoc_hook:nn #1#2
{ \ztocgroupinsert{#1}{\fbox{\color{#2}\sffamily #1}} }

% begin/end hooks:
\__debug_ztoc_hook:nn {subsection,1,begin}{red}
\__debug_ztoc_hook:nn {subsection,1,end}{red}
\__debug_ztoc_hook:nn {subsubsection,5,begin}{gray}
\__debug_ztoc_hook:nn {subsubsection,5,end}{gray}
```

例 2

```
% new before/after hooks:
```

```
\__debug_ztoc_hook:nn {subsection,1,before}{blue}
\__debug_ztoc_hook:nn {subsection,1,after}{blue}
\__debug_ztoc_hook:nn {subsubsection,1,before}{teal}
\__debug_ztoc_hook:nn {subsubsection,1,after}{teal}
\__debug_ztoc_hook:nn {subsubsection,3,before}{purple}
\__debug_ztoc_hook:nn {subsubsection,3,after}{purple}
\ExplSyntaxOff
\zlocaltoc{subsection}{4}
```

subsection,1,before	BG:0	
6.1 font 模块		14
EG:0 subsection,1,begin	subsubsection,1,before	BG:1
6.1.1 字体机制		14
EG:1 subsubsection,1,after	BG:2	
6.1.2 默认字体族		17
EG:2 subsubsection,3,before	BG:3	
6.1.3 新建字体族		17
EG:3 subsubsection,3,after	BG:4	
6.1.4 切换字体		19
EG:4	BG:5	
6.1.5 Xe _{La} TeX 接口		20
EG:5 subsubsection,5,begin	subsubsection,5,end	BG:6
6.1.6 杂项		24
EG:6 subsection,1,end	subsection,1,after	

5 案例:3

这个案例相较之上一个案例更加复杂:

```
\begin{group}\makeatletter
\tocupdatelinewidthfix
\def\customtothemecolor{teal}
\ztocformat\subsection
{
  explicit=true,
  code={%
    \noindent\fcolorbox{white}{\customtothemecolor}{\hb@xt@{\linewidthfix
      {\bfseries\large\color{white}\ztocname
        \hskip.75em\relax\ztoctitle\hfill\ztocpage}}}%
  }
}
\ztocformat\subsubsection
{
  format+=\color{\customtothemecolor},
  page.format+=\color{\customtothemecolor},
  name.before=\S\;
}
\ztocgroupinsert{subsection,1,begin}%
{%
  \begin{tcolorbox}[
    enhanced, sharp corners,
    boxrule=.5pt, colback=white,
    left=20pt, colframe=\customtothemecolor,
    overlay={
      \ornamentcorner{north west}{north west}{0}
      \ornamentcorner{north west}{south west}{90}
      \ornamentcorner{north west}{south east}{180}
      \ornamentcorner{north west}{north east}{270}
    }
  ]
```

例 3

```
]
\pgfornament[width = 2.5cm,color = \customtothemecolor]{8}%
\kern-6pt\relax\begin{minipage}{.75\linewidth}%
}
\ztcgroupinsert{subsection,1,end}%
{%
\end{minipage}%
\end{tcolorbox}%
}
\zlocaltoc{subsection}{4}
\makeatother\endgroup
```

6.1 font 模块		14
	§ 6.1.1 字体机制	14
	§ 6.1.2 默认字体族	17
	§ 6.1.3 新建字体族	17
	§ 6.1.4 切换字体	19
	§ 6.1.5 𐤀T𐤁X 接口	20
	§ 6.1.6 杂项	24

6 案例:4

我们可以使用 `\ztocgroupinsert` 命令复刻 `titletoc` 宏包所提供的样式:

```
\makeatletter
\def\customssstyle#1{%
  \ztocgroupinsert{subsection,#1,begin}
    {\raggedright\hangindent2.3em\relax\noindent\hskip2.3em\relax 【}
  \ztocgroupinsert{subsection,#1,end}{】 \par}
}
\makeatother
\customssstyle{1} \customssstyle{2}
\customssstyle{3} \customssstyle{4}
{
  \ztocformat\subsubsection{
    explicit=true,
    code={%
      \textit{\ztocname\enskip\ztoctitle\enskip
        (\ztochyperpage{\ztocpage})\ztociflast{}{;} \enskip}}
    }
  \ztocformat\subsection{
    explicit=true,
    code = {%
      \S\;\ztocname\enskip\ztoctitle
      \hskip1em\ztochyperpage{\ztocpage}\par}
    }
  \begin{multicols}{2}
    \zlocaltoc{subsection}{4, 11, 8, 6}
  \end{multicols}
}
```

例 4

§ 6.1 font 模块 14

【6.1.1 字体机制 (14); 6.1.2 默认字体族 (17); 6.1.3 新建字体族 (17); 6.1.4 切换字体 (19); 6.1.5 $\text{\LaTeX}$ 接口 (20); 6.1.6 杂项 (24)】

§ 6.8 sect 模块 83

【6.8.1 序言 (83); 6.8.2 层级/模板 (84); 6.8.3 章节标题 (85); 6.8.4 章节目录 (91); 6.8.5 使用案例 (105); 6.8.6 编程接口: 初始化 (131); 6.8.7 编程接口: 章节命令 (134); 6.8.8 编程接口: 目

录 (137); 6.8.9 编程接口: 模板 (150); 6.8.10 编程接口: 杂项 (152)】

§ 6.5 thm 模块 41

【6.5.1 用户接口 (42); 6.5.2 定理目录 (48); 6.5.3 高级接口 (51); 6.5.4 环境钩子 (55)】

§ 6.3 page 模块 31

【6.3.1 页面布局 (31); 6.3.2 页眉页脚 (32); 6.3.3 页面水印 (35); 6.3.4 杂项 (37)】

7 案例: 5

这个案例相较之上一个案例更加复杂:

```
\begin{group}\makeatletter
% \setcounter{tocdepth}{3}
\def\customtothemecolor{SeaGreen}
\ztcformat\section
{
  explicit=true,
  code={%
    \noindent\hskip1.5em\fbcolorbox{white}{\customtothemecolor}{\hb@xt@
      \dimexpr\linewidth-2\fbboxsep-1.5em\relax
      {\bfseries\large\color{white}第\zhnumber{\ztocname}节
        \hskip.75em\relax\ztoctitle\hfill\ztocpage}}}%
  }
}
\ztcformat\subsection{
  name.before=\S\;, name.width=2.7em,
  format+=\color{\customtothemecolor},
  page.format+=\color{\customtothemecolor},
  leader.sep=2pt, page.width=1em,
  hyper.title=true,
  leader.content=\textcolor{\customtothemecolor}{.},
}
\ztcgroupinsert{section,1,before}%
{%
  \begin{tcolorbox}[
    enhanced, sharp corners,
    boxrule=.5pt, colback=white,
    left=20pt, colframe=\customtothemecolor,
    overlay={
      \ornamentcorner{north west}{north west}{0}
      \ornamentcorner{north west}{south west}{90}
    }
  ]

```

例 5

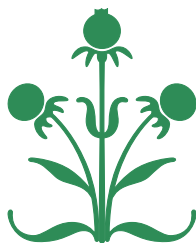
```

\ornamentcorner{north west}{south east}{180}
\ornamentcorner{north west}{north east}{270}
}
]
\pgfornament[width = 2.5cm,color = \customtocthemecolor]{10}%
\begin{minipage}{.77\linewidth}%
}
\ztcgroupinsert{section,1,begin}{%
% \setlength{\columnsep}{0em}
\begin{multicols}{2}}
\ztcgroupinsert{section,1,after}{%
{%
\end{multicols}%
\end{minipage}%
\end{tcolorbox}%
}
\zlocaltoc{section}{6}
\makeatother\endgroup

```

第六节 $\LaTeX$ 模块

13



§ 6.1 font 模块	14	§ 6.7 cmd 模块	69
§ 6.2 ref 模块	25	§ 6.8 sect 模块	83
§ 6.3 page 模块	31	§ 6.9 sclist 模块	154
§ 6.4 color 模块	38	§ 6.10 graphics 模块	158
§ 6.5 thm 模块	41	§ 6.11 counter 模块	159
§ 6.6 box 模块	59		

8 案例: 6

这个案例复刻了 etoc 中的目录样式 (参见 etoc 宏包第 39 节):

```

1 \ztcgroupinsert{section,1,before} 1
2   {\begin{longtable}{|l|l|r|}} 2
3 \ztcgroupinsert{section,4,after} 3
4   {\ \hline\end{longtable}} 4
5 { 5
6   \ztcformat\section{ 6
7     explicit=true, 7
8     code = {% 8
9       \ztciffirst{\kill\hline\multicolumn{3}{|c|}{\large\bfseries Table of 9
Contents}\ \hline}{\ \hline}%
10       \multicolumn{3}{|c|}{\bfseries 第\;\ztocname\;节\enskip\ztoctitle}% 10
11     } 11
12   } 12
13 \ztcformat\subsection{ 13
14   explicit=true, 14
15   code = {\ \hline 15
16     \textbf{\S\ztocname} & \ztocname\enskip\ztoctitle & \ztohyperpage{ 16
\ztocpage}
17   } 17
18 } 18
19 \ztcformat\subsubsection{ 19
20   explicit=true, 20
21   code = {\ \ 21
22     & \ztocname\enskip\ztoctitle & \ztohyperpage{\ztocpage} 22
23   } 23
24 } 24
25 \zlocaltoc{section}{1, 2, 7, 9} 25
26 } 26

```

Table of Contents

第 1 节 基本介绍		
第 2 节 安装使用		
§2.1	2.1 在线模板	2
§2.2	2.2 本地安装	2
§2.3	2.3 快速开始	3
第 7 节 $\LaTeX$ 库		
§7.1	7.1 fancy 库	161
§7.3	7.3 alias 库	167
	7.3.1 数学字体	168
	7.3.2 数学箭头	169
	7.3.3 其它符号	173
	7.3.4 数学算子	175
	7.3.5 自动括号	177
	7.3.6 微分算子	178
	7.3.7 矩阵	178
	7.3.8 编程接口	186
§7.4	7.4 slide 库	190
	7.4.1 颜色主题	191
	7.4.2 页面信息	193
	7.4.3 编程接口	197
§7.5	7.5 thm 库	203
第 9 节 TODO		

9 案例: 7

这个案例也是受到 etoc 宏包的启发 – 将目录展示为思维导图的形式:

```

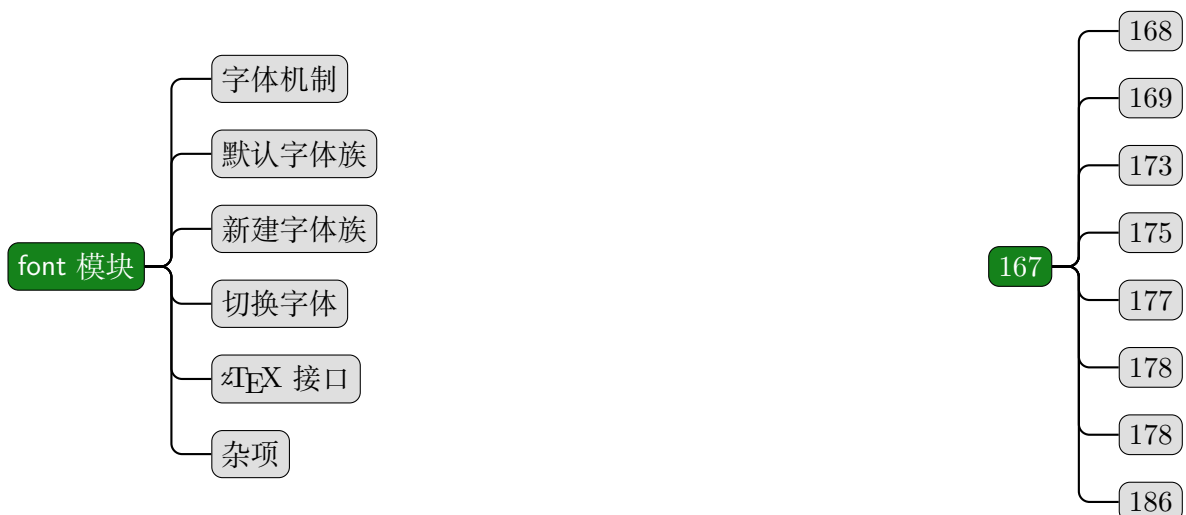
1 \definecolor{Green}{HTML}{15821b} 1
2 \tikzset{ 2
3   rootKey/.style={draw, rectangle, rounded corners, fill=Green, text=white}, 3
4 } 4
5 \ExplSyntaxOn 5
6 \cs_new:Npn \__toc_tree:nn #1#2 6
7 { 7
8   \quark_if_no_value:nF {#1} 8
9   { 9
10     \Tree[.\node[rootKey]{#1};~#2 ] 10
11   } 11
12 } 12
13 \cs_generate_variant:Nn \__toc_tree:nn {oe} 13
14 \def\wrapper#1[. {\exp_not:n {#1}}~] 14
15 \cs_new:Npn \ztoc_typeset_toc tree:nnn #1#2#3 15
16 { 16
17   \ztoc_table_filter_key_byclass:nnnNN {#1}{#2}{#3} 17
18   \g_ztoc_keyvaltoc_seq \g_tmpb_seq 18
19   \seq_gpop_left:NN \g_tmpb_seq \secRoot 19
20   \__toc_tree:oe { \secRoot } 20
21   { \seq_map_function:NN \g_tmpb_seq \wrapper } 21
22 } 22
23 \def\TocTree#1#2#3 23
24 { 24
25   \exp_args:Nne \ztoc_typeset_toc tree:nnn 25
26     {#1}{#2}{#3} 26
27 } 27
28 \ExplSyntaxOff 28
29 29
30 \noindent\begin{tikzpicture}[ 30

```

```

31     grow'=right, 31
32     level distance=50pt, 32
33     sibling distance=10pt, 33
34     level 1/.style={level distance=25pt}, 34
35     every tree node/.style={anchor=west, draw, rectangle, rounded corners, 35
fill=gray!25},
36     every level 0 node/.style={anchor=east}, 36
37     edge from parent/.style={ 37
38         draw, thick, rounded corners, 38
39         edge from parent path={ 39
40             (\tikzparentnode.east) -- +(10pt, 0pt) 40
41             |- (\tikzchildnode.west) 41
42         } 42
43     }, 43
44 ] 44
45 \TocTree{subsection}{4}{title} 45
46 \begin{scope}[xshift=12cm] 46
47     \TocTree{subsection} 47
48     {\ztocindex{subsection}{\pkg{alias} 库}} 48
49     {page} 49
50 \end{scope} 50
51 \end{tikzpicture} 51

```



10 案例: 8

这个案例相较之上一个案例更加复杂, 它展示了 `\ztoc_group_parser:NNNnnnnn` 命令中 `\step code` 的使用方法:

```

1 % \usetikzlibrary { mindmap, trees } 1
2 \ExplSyntaxOn 2
3 \int_new:N \g__mindmap_color_int 3
4 \seq_new:N \g__mindmap_data_seq 4
5 \seq_new:N \g__mindmap_keyvaldata_seq 5
6 \tl_new:N \g__mindmap_filter_data_tl 6
7 \def\toclinecmd#1#2#3#4 7
8 { 8
9   % override hyperlink color. 9
10   {\ztexlink{#1.#2}{\textcolor{white}{#3(#4)}}} 10
11 } 11
12 \cs_new:Npn \__ztoc_gen_mindmap_data:nn #1#2 12
13 { 13
14   \tl_gclear:N \g_tmpa_tl 14
15   \ztoc_table_filter_byclass:nnNN 15
16   { #1 }{ #2 } 16
17   \g_ztoc_keyvaltoc_seq 17
18   \g__mindmap_data_seq 18
19   \ztoc_generate_keyvaltable_seq:NN 19
20   \g__mindmap_data_seq 20
21   \g__mindmap_keyvaldata_seq 21
22   \ztoc_group_parser:NNNnnnnn \g__mindmap_keyvaldata_seq 22
23   \toclinecmd \g_tmpa_tl 23
24   { 24
25     child [ concept~color = blue! 25
26       \int_eval:n{\g__mindmap_color_int*6}!green 26
27     ] \bgroup node[concept] 27
28   } 28
29   {}{}{\egroup}{\int_gincr:N \g__mindmap_color_int} 29

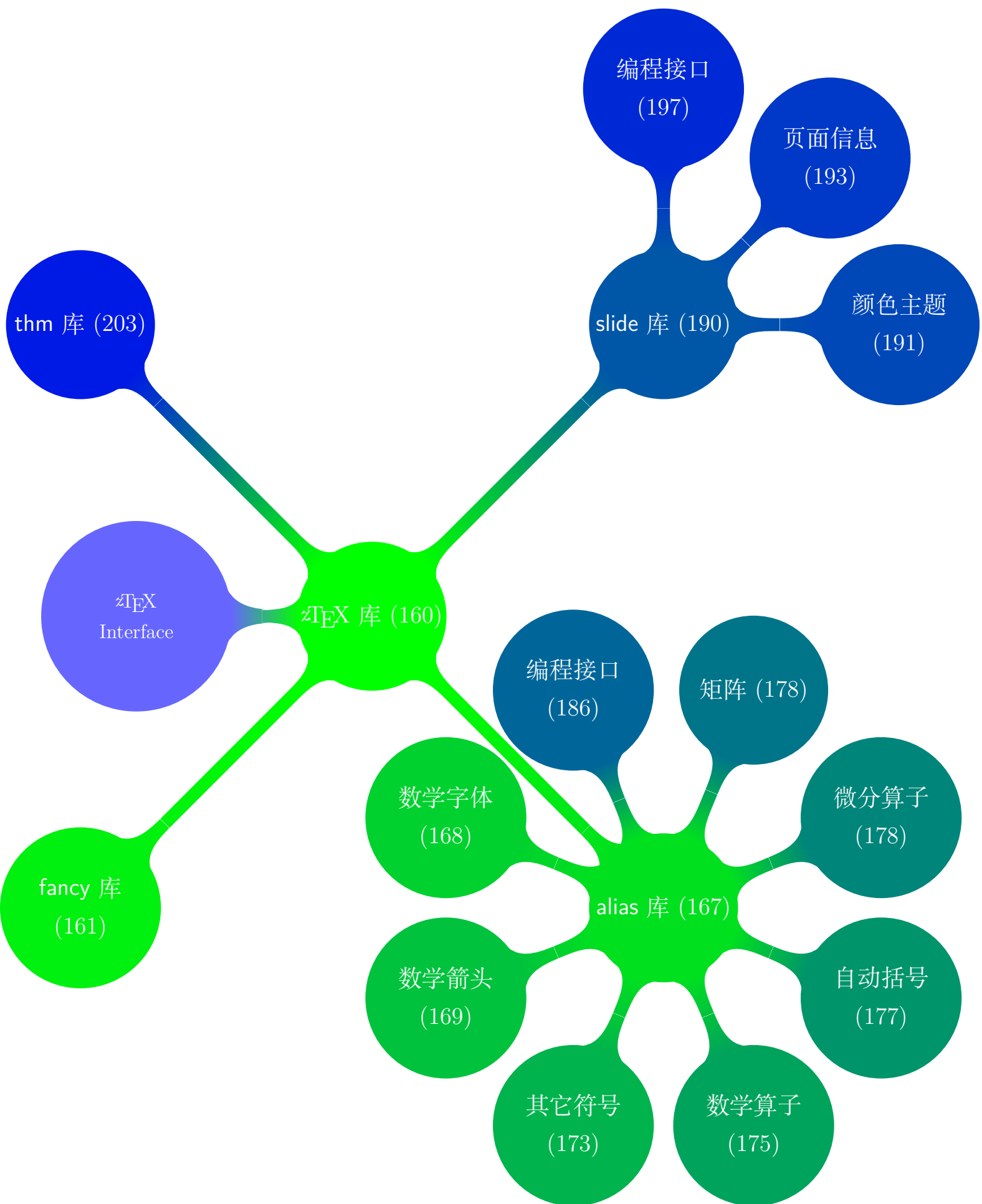
```

```

30 % replace '\bgroup' and '\egroup' by '{' and '}'.
31 \str_clear:N \l_tmpa_str
32 \exp_args:NNo \str_set:Nn \l_tmpa_str { \g_tmpa_tl }
33 \exp_args:NNne \str_replace_all:Nnn \l_tmpa_str
34 { \bgroup }{ \char_generate:nn {123}{12} }
35 \exp_args:NNne \str_replace_all:Nnn \l_tmpa_str
36 { \egroup }{ \char_generate:nn {125}{12} }
37 \tl_gset_rescan:Nno \g__mindmap_filter_data_tl
38 { \cctab_select:N \c_document_cctab }
39 { \l_tmpa_str }
40 }
41 \cs_new:Npn \__tkz_mind_map:n #1
42 {
43   \tl_if_empty:nF {#1}
44   {
45     \node[concept]
46       {\color{white}\ztext{}~Interface}
47       #1;
48   }
49 }
50 \newcommand{\mindmap}[2]
51 {
52   \__ztoc_gen_mindmap_data:nn
53     { #1 }{ #2 }
54   \exp_args:No \__tkz_mind_map:n
55     { \g__mindmap_filter_data_tl }
56 }
57 \ExplSyntaxOff
58 \begin{tikzpicture}[
59   mindmap, grow cyclic,
60   text width=2cm, align=flush center,
61   nodes={concept}, concept color=blue!60,
62   root concept/.append style={

```

```
63      text width=4cm, font=\Large 63
64  }, 64
65  level 1 concept/.append style={ 65
66      font=\normalsize, color=white 66
67  }, 67
68  level 1/.append style={ 68
69      level distance=5cm, sibling angle=45, 69
70      text width=3cm, font=\Large 70
71  }, 71
72  level 2/.append style={ 72
73      level distance=9cm, sibling angle=90, 73
74      text width=3cm, font=\Large 74
75  }, 75
76  level 3/.append style={ 76
77      level distance=5cm, sibling angle=45, 77
78      text width=3cm, font=\Large 78
79  }, 79
80  ] 80
81  \mindmap{section}{7} 81
82  \end{tikzpicture} 82
```



11 案例: 9

下面这个案例展示了 `\ztociffirst`, `\ztociflast` 以及 `\ztocdepth` 等命令的使用方法:

```

1  \ExplSyntaxOn
2  \newcommand{\nbraket}
3    { \makebox[0pt]{\rule[-7.5pt]{.75pt}{19pt}} }
4  \newcommand{\ubraket}
5    {
6    \makebox[0pt]{
7      \rule[-7pt]{.75pt}{10pt}
8    }
9    \makebox[0pt][l]{
10     \kern-.375pt\raise3pt\hbox{\rule{3pt}{.75pt}}
11   }
12 }
13 \newcommand{\bbraket}
14 {
15   \makebox[0pt]{ \rule[3pt]{.75pt}{10pt} }
16   \makebox[0pt][l]
17     { \kern-.375pt\raise3pt
18       \hbox{\rule{3pt}{.75pt}} }
19 }
20 \newcommand{\showbar}
21 {
22   \prg_replicate:nn {\ztocdepth - 2}
23     { \kern20pt \nbraket }
24   \kern20pt \ztociffirst { \ubraket }
25     { \ztociflast{\bbraket}{\nbraket} }
26 }
27 \ztocformat{\section, \subsection, \subsubsection}
28 {
29   explicit=true,
30   code = {

```

```
31      \noindent                                     31
32      \prg_replicate:nn {\ztocdepth-1}              32
33      {                                              33
34          \hskip 20pt\relax                          34
35      }                                              35
36      \llap{\smash{\showbar}}                      36
37      \kern10pt                                     37
38      \ztoctitle\par                                38
39  }                                              39
40 }                                              40
41 \ExplSyntaxOff                                    41
42 {                                              42
43     \ztocformat\subsection{ignore.name={10.}}      43
44     \ztocformat\subsubsection{ignore.name={10.}}  44
45     \multicolcontents{2}                          45
46 }                                              46
```

- 基本介绍
- 安装使用
 - 在线模板
 - 本地安装
 - 快速开始
- 基本命令
- 文档类选项
- 状态检测
- $\LaTeX$ 模块
 - font 模块
 - 字体机制
 - 默认字体族
 - 新建字体族
 - 切换字体
 - $\LaTeX$ 接口
 - 杂项
 - ref 模块
 - 超链接
 - 标签与引用
 - 杂项
 - page 模块
 - 页面布局
 - 页眉页脚
 - 页面水印
 - 杂项
 - color 模块
 - thm 模块
 - 用户接口
 - 定理目录
 - 高级接口
 - 环境钩子
 - box 模块
 - cmd 模块
 - 列表补丁
 - token 命令

- sect 模块
 - 序言
 - 层级/模板
 - 章节标题
 - 章节目录
 - 使用案例
 - 编程接口: 初始化
 - 编程接口: 章节命令
 - 编程接口: 目录
 - 编程接口: 模板
 - 编程接口: 杂项
- sclist 模块
- graphics 模块
- counter 模块
- $\LaTeX$ 库
 - fancy 库
 - alias 库
 - 数学字体
 - 数学箭头
 - 其它符号
 - 数学算子
 - 自动括号
 - 微分算子
 - 矩阵
 - 编程接口
 - slide 库
 - 颜色主题
 - 页面信息
 - 编程接口
 - thm 库
- ztool 宏包
- TODO
- $\LaTeX$ 源码
- 索引

12 案例: 10

下面这个案例相较之上一个案例更加复杂, 但它展示了 `\ztoc_group_parser:NNNnnnnn` 命令的强大和普适性:

```

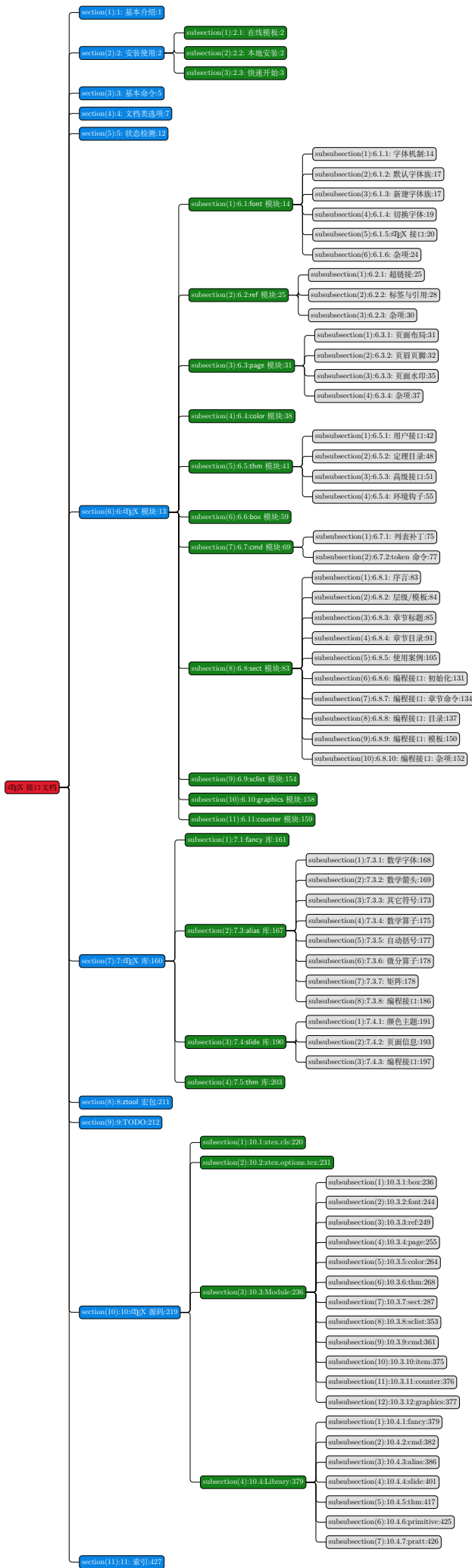
1 % \usepackage{tikz} 1
2 % \usepackage{tikz-qtrees} 2
3 % \usepackage{tikz-qtrees-patch} 3
4 \definecolor{Red}{HTML}{e32b2d} 4
5 \definecolor{Green}{HTML}{15821b} 5
6 \definecolor{Blue}{HTML}{0984e3} 6
7 \tikzset{ 7
8   section/.style={draw, rectangle, rounded corners, fill=Red}, 8
9   subsection/.style={draw, rectangle, rounded corners, fill=Blue, text=white}, 9
10  subsubsection/.style={draw, rectangle, rounded corners, fill=Green, 10
    text=white},
11 } 11
12 \ExplSyntaxOn 12
13 % \def\toclinecmd#1#2#3#4{{#1:#2:#3:#4}~} 13
14 \def\toclinecmd#1#2#3#4{{\gparserclass(\gparserindex):#2:#3:#4}~} 14
15 \ztoc_group_parser:NNNnnnnn \g_ztoc_keyvaltoc_seq 15
16 \toclinecmd \g_tmpa_tl {[.]{}}{~}]{} 16
17 \def\TocTree 17
18 { 18
19   \exp_args:Nno \__toc_tree:nn 19
20   {\ztex{}~接口文档} 20
21   {\g_tmpa_tl} 21
22 } 22
23 \cs_new:Npn \__toc_tree:nn #1#2 23
24 { 24
25   \quark_if_no_value:nF {#1} 25
26   { 26
27     \Tree[.\node[section]{#1};~#2 ] 27
28   } 28

```

```

29   }
30   \ExplSyntaxOff
31
32   \thispagestyle{empty}
33   \begin{tikzpicture}[
34       grow'=right, level distance=1cm,
35       sibling distance=10pt,
36       every tree node/.style = {
37           anchor=west, outer sep=0pt, draw, rectangle,
38           rounded corners, fill=gray!25
39       },
40       every level 1 node/.style = {
41           subsection
42       },
43       every level 2 node/.style = {
44           subsubsection
45       },
46       edge from parent/.style={
47           draw, thick, rounded corners,
48           edge from parent path={
49               (\tikzparentnode.east) -- +(.5cm, 0pt)
50               |- (\tikzchildnode.west)
51           },
52       },
53   ]
54   \TocTree
55   \end{tikzpicture}

```



TikZ Examples

Eureka

2025 年 5 月 31 日

总目录

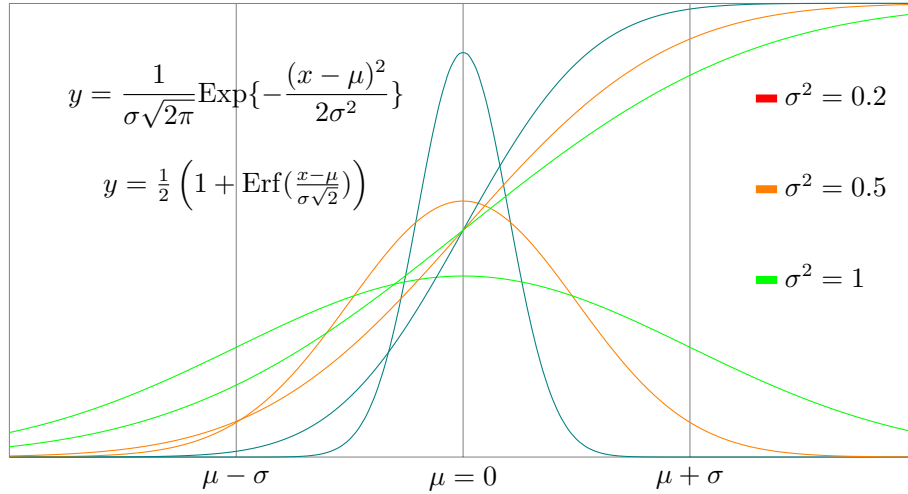
1	介绍	3	3.3	案例 11	14
			3.4	案例 12	15
2	basic/gnuplot 库	4	3.5	案例 13	16
2.1	案例 1	4	3.6	案例 14	17
2.2	案例 2	5	3.7	案例 15	18
2.3	案例 3	6			
2.4	案例 4	7	4	python 库	19
2.5	案例 5	8	4.1	案例 16	19
2.6	案例 6	9	4.2	案例 17	20
2.7	案例 7	10	4.3	案例 18	21
2.8	案例 8	11			
3	wolfram 库	12	5	l3draw 库	23
3.1	案例 9	12	5.1	案例 19	23
3.2	案例 10	13	5.2	案例 20	24
			5.3	案例 21	25

1 介绍

本文档展示了 `\tikz` 宏包中部分命令或环境的使用示例, 希望本文档可以帮助用户更好的掌握与使用 `\tikz` 宏集.

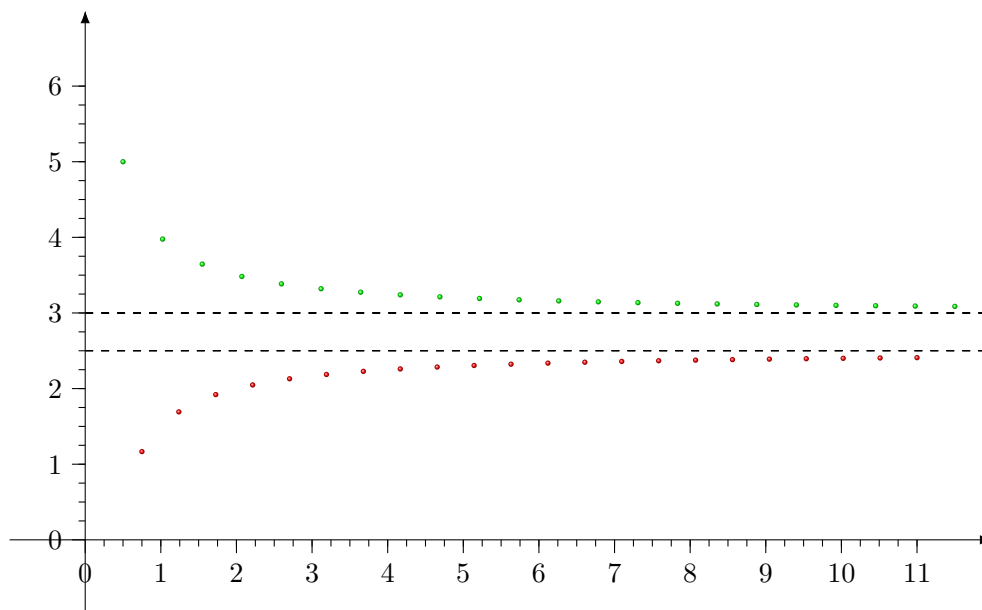
2 basic/gnuplot 库

2.1 案例 1



```
\begin{tikzpicture}[yscale=6, xscale=3]
  \ShowGrid{(-2,0); (2,1)}
  % pdf
  \Plot[domain=-2:2,style=teal]{1/(sqrt(0.2)*sqrt(2*pi))*exp(-(x-0)**2/(2*0.2**2))}
  \Plot[domain=-2:2,style=orange]{1/(sqrt(0.5)*sqrt(2*pi))*exp(-(x-0)**2/(2*0.5**2))}
  \Plot[domain=-2:2,style=green]{1/(sqrt(1)*sqrt(2*pi))*exp(-(x-0)**2/(2*1**2))}
  % cdf
  \Plot[domain=-2:2,style=teal]{0.5*(1+erf((x-0)/(sqrt(0.2)*sqrt(2))))}
  \Plot[domain=-2:2,style=orange]{0.5*(1+erf((x-0)/(sqrt(0.5)*sqrt(2))))}
  \Plot[domain=-2:2,style=green]{0.5*(1+erf((x-0)/(sqrt(1)*sqrt(2))))}
  % annotate
  \ShowPoint[radius=0pt]{(-1, 0); (0, 0); (1, 0)}
  [$\mu-\sigma$, $\mu=0$, $\mu+\sigma$][below]
  \ShowPoint[radius=0pt]{(1, 0.8); (1, 0.6); (1, 0.4)}[
    \textcolor{red}{\rule[1pt]{8pt}{3pt}}\;$\sigma^2=0.2$;
    \textcolor{orange}{\rule[1pt]{8pt}{3pt}}\;$\sigma^2=0.5$;
    \textcolor{green}{\rule[1pt]{8pt}{3pt}}\;$\sigma^2=1$;
  ][right=2em]
  \ShowPoint[radius=0pt]
  {(-1, 0.8); (-1, 0.6)}
  [
    $\displaystyle y = \frac{1}{\sigma\sqrt{2\pi}}\mathrm{Exp}$
    $\{-\frac{(x-\mu)^2}{2\sigma^2}\}$;
    $y=\frac{1}{2}\left(1+\mathrm{Erf}\left(\frac{x-\mu}{\sigma\sqrt{2}}\right)\right)$
  ]
\end{tikzpicture}
```

2.2 案例 2

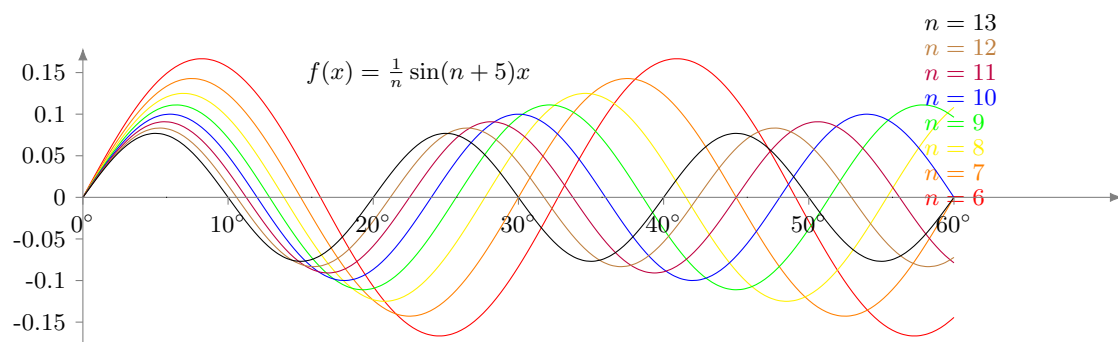


```

\begin{tikzpicture}[>=Latex]
  \xAxis[-1][12] \yAxis[-1][7]
  \PlotPrecise{plot}{22}
  \Plot[
    domain=0.75:11,
    style={red, thick, opacity=0},
    marker={type=ball, color=red}
  ]{2.5-1/x}
  \PlotPrecise{plot}{22}
  \Plot[
    domain=0.5:11.5,
    style={red, thick, opacity=0},
    marker={type=ball, color=green}
  ]{3+1/x}
  \PlotPrecise*{contour}{40}
  \ContourPlot[domain=0:12;, style={dashed}]{y-2.5}
  \ContourPlot[domain=0:12;, style={dashed}]{y-3}
\end{tikzpicture}

```

2.3 案例 3

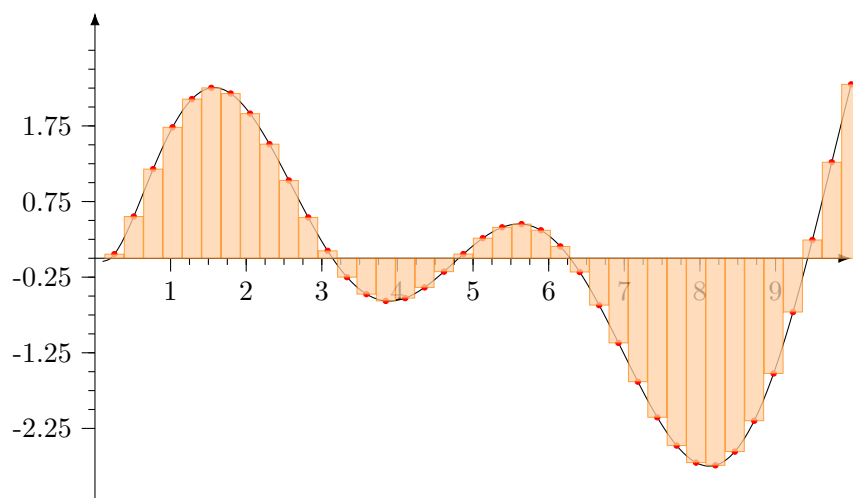


```

\ExplSyntaxOn
\clist_new:N \l__color_clist
\clist_set:Nn \l__color_clist {red, orange, yellow, green, blue, purple, brown, black}
\newcommand{\colorItem}[1]{\clist_item:Nn \l__color_clist {#1}}
\def\fpToint#1{\fp_to_int:n {#1}}
\ExplSyntaxOff
\begin{tikzpicture}[scale=11, >=Latex, font=\small]
  % plot and annotate
  \node at (.55, 0.15) [left] {$f(x)=\frac{1}{n}\sin(n+5)x$};
  \foreach \i in {6, 7, 8, 9, 10, 11, 12, 13}{
    \Plot[
      domain=0:pi/3,
      style=\colorItem{\fpeval{\i-5}}
    ]{\fpeval{1/\i}*sin(\fpeval{\i+5}*x)}
    \node[color=\colorItem{\fpeval{\i-5}}]
      at (1, \fpeval{(\i-6)*0.03}) [right] {$n=\i$};
  }
  % axis draw
  \ShowAxis [
    tickStyle=above,    axisColor=gray,
    tickStart=-0.15,    tickEnd=0.18,
    mainStep=0.05,
    mainTickColor=gray, mainTickLabelPosition=left,
    mainTickLength=.5pt, axisRotate=90,
  ]{(-0.18, 0); (0.18, 0)}
  \ShowAxis [
    tickStyle=below,    axisColor=gray,
    tickStart=0,        tickEnd=1.22,
    mainStep=\fpeval{pi/18},
    mainTickColor=gray, subTickLength=0pt,
    mainTickLength=.5pt,
    mainTickLabel={\fpToint{\CurrentFp/(pi/18)*10}$^\circ$}
  ]{(0, 0); (1.25, -0)}
\end{tikzpicture}

```

2.4 案例 4

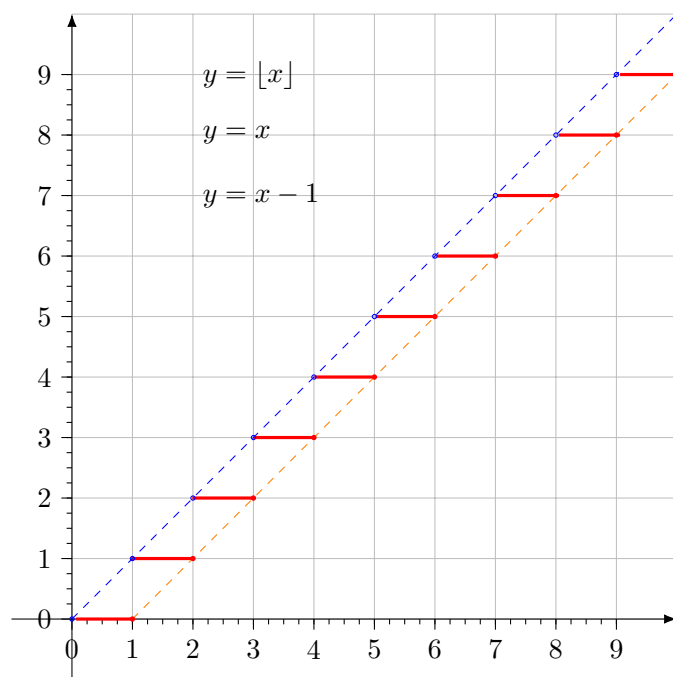


```

\begin{tikzpicture}[>=Latex]
  \xAxis[0][10] \yAxis[-3.25][3.25]
  \Plot[domain=0:10]{2*sqrt(x)*cos(log(x))*sin(x)}
  \PlotPrecise{plot}{40}
  \Plot[
    domain=0:10, style={opacity=0},
    marker={type=*, color=red}
  ]{2*sqrt(x)*cos(log(x))*sin(x)}
  \BarPlot[x][
    fill=orange!35!white,
    bar width=\fpeval{10/40}cm,
    opacity=.75, very thin, draw=orange
  ]{\gnudata{2}}
\end{tikzpicture}

```

2.5 案例 5

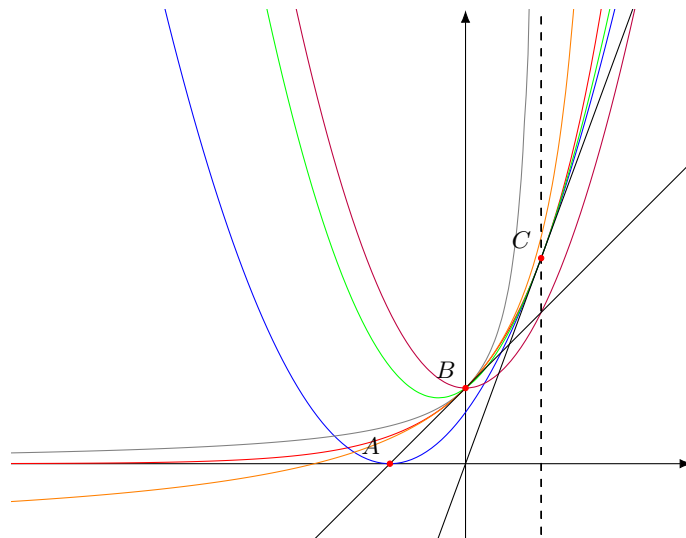


```

\begin{tikzpicture}[scale=.8, >=Latex]
  \ShowGrid[step=1, color=gray, opacity=.5]{(0, 0); (10, 10)}
  \xAxis[-1][10] \yAxis[-1][10]
  \Plot[
    domain=0:10,
    style={red, jump mark right, very thick, xshift=2pt},
    marker={type=*, opacity=0}
  ]{\floor(x)}
  \Plot[domain=0:10, style={dashed, blue}]{x}
  \Plot[domain=1:10, style={dashed, orange}]{x-1}
  \PlotPrecise{plot}{11}
  \Plot[
    domain=0:10,
    style={opacity=0, jump mark right},
    marker={type=o, color=blue}
  ]{x}
  \PlotPrecise{plot}{11}
  \Plot[
    domain=0:10,
    style={opacity=0, jump mark right},
    marker={type=*, color=red}
  ]{x-1}
  \ShowPoint[opacity=0]{(2, 9); (2, 8); (2, 7)}
  [$y=\lfloor x \rfloor$; $y=x$; $y=x-1$][right]
\end{tikzpicture}

```

2.6 案例 6

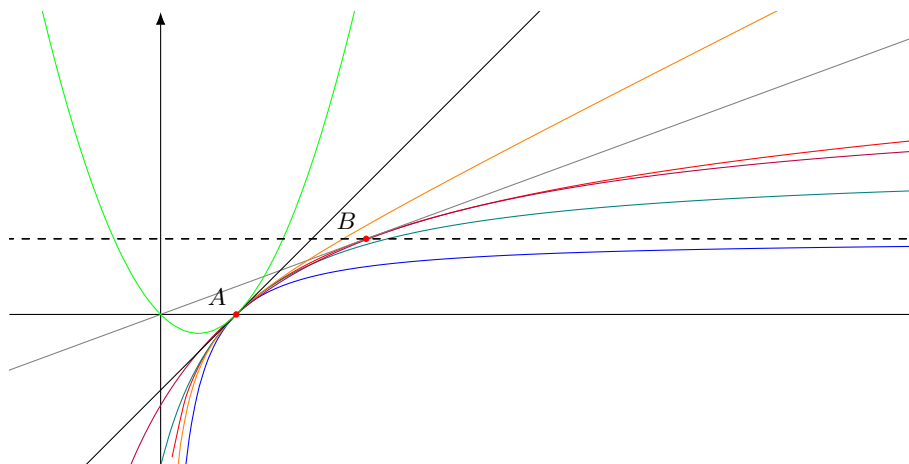


```

\begin{tikzpicture}[>=Latex, font=\small]
  \clip (-6, -1) rectangle (3, 6);
  \ShowAxis[(-8, 0); (3, 0)] \ShowAxis[(0, -1.5); (0, 6)]
  \Plot[domain=-8:5, style={red}] {exp(x)}
  \Plot[domain=-8:5, style={blue}] {exp(1)/4*(x+1)**2}
  \Plot[domain=-8:5, style={green}] {exp(1)*x + (x-1)**2}
  \Plot[domain=-8:5, style={purple}] {x**2 + 1}
  \Plot[domain=-8:0.95, style={gray}] {1/(1-x)}
  \Plot[domain=-8:1.95, style={orange}] {(2+x)/(2-x)}
  \Plot[domain=-8:5] {x+1}
  \Plot[domain=-8:8] {exp(1)*x}
  \ContourPlot[domain={0:2;-6:6}, style=dashed] {x-1}
  \ShowPoint[color=red, radius=1pt]{(-1, 0); (0, 1); (1, 2.71828)}
  [A$, B$, C$][above left]
\end{tikzpicture}

```

2.7 案例 7

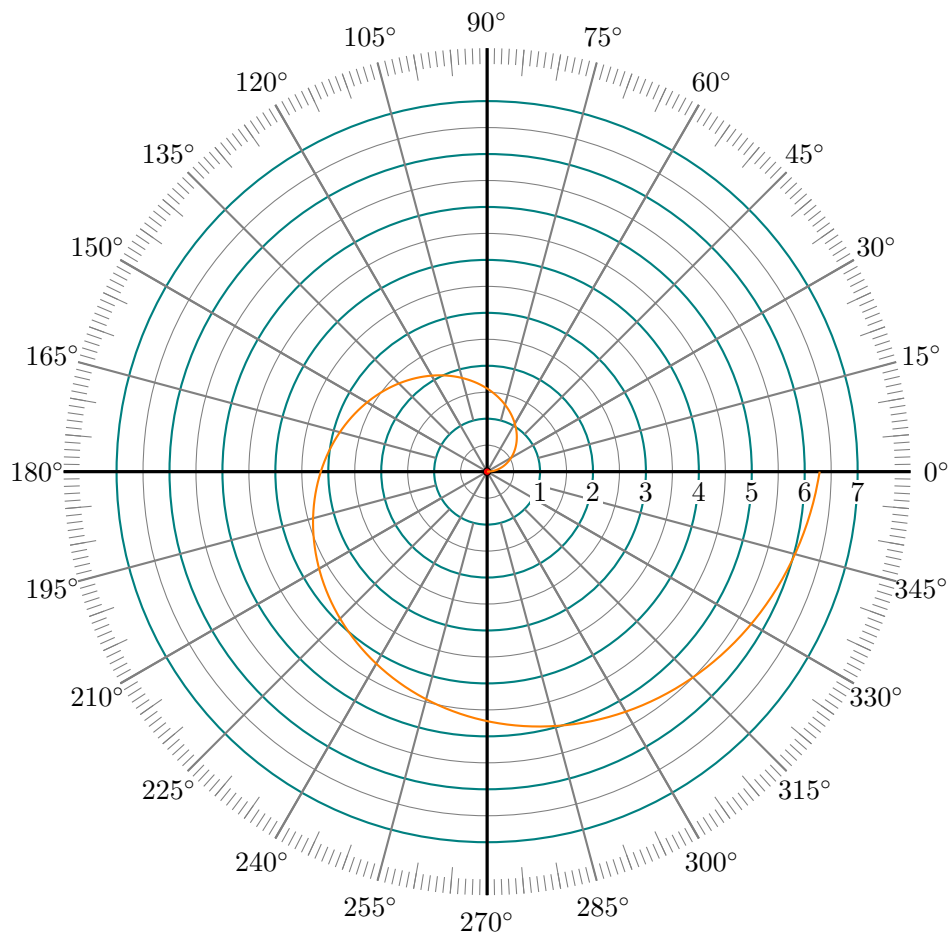


```

\begin{tikzpicture}[>=Latex, font=\small]
  \clip (-2, -2) rectangle (10, 4);
  \ShowAxis{(-2, 0); (12, 0)} \ShowAxis{(0, -2); (0, 4)}
  \Plot[domain=-5:12, style={red}]      {\log(x)}
  \Plot[domain=0:12, style={blue}]      {(x-1)/x}
  \Plot[domain=0:12, style={teal}]      {2*(x-1)/(x+1)}
  \Plot[domain=-1:12, style={purple}]   {6*(x-1)/(2*x+5)}
  \Plot[domain=-5:12, style={gray}]     {x/exp(1)}
  \Plot[domain=0.1:12, style={orange}]  {0.5*(x-1/x)}
  \Plot[domain=-5:12]                   {x-1}
  \Plot[domain=-5:12, style=green]      {x**2-x}
  \ContourPlot[domain={-5:12;-6:6}, style=dashed]{y-1}
  \ShowPoint[color=red, radius=1pt]{(1, 0);(2.71828, 1)}
  [A$, B$][above left]
\end{tikzpicture}

```

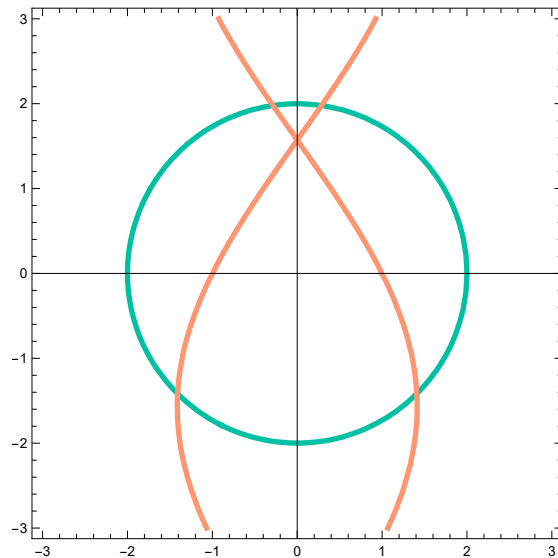
2.8 案例 8



```
% https://texample.net/tikz/examples/polar-coordinates-template/
\begin{tikzpicture}[scale=.7]
  \foreach \r in {1, 2,...,7} \draw[teal,thick] (0,0) circle (\r);
  \foreach \r in {0.5, 1.5,...,7} \draw[gray, thin] (0,0) circle (\r);
  \foreach \a in {0, 1,...,359} \draw[gray] (\a:7.7) -- (\a:8);
  \foreach \a in {0, 5,...,359} \draw[gray] (\a:7.5) -- (\a:8);
  \foreach \a in {0, 15,...,359} \draw[thick,gray] (\a:1) -- (\a:8);
  \foreach \a in {0, 30,...,359} \draw[thick,gray] (0, 0) -- (\a:8);
  \foreach \r in {1, 2,...,7}
    \draw (\r,0) node[inner sep=1pt,below=3pt,rectangle,fill=white] {$\r$};
  \foreach \a in {0, 90,...,359} \draw[very thick] (0, 0) -- (\a:8);
  \foreach \a in {0, 15,...,359} \draw (\a: 8.5) node {$\a^\circ$};
  \draw[fill=red] (0,0) circle(0.7mm);
  \PolarPlot[domain=0:2*pi, style={thick, orange}]{t}
\end{tikzpicture}
```

3 wolfram 库

3.1 案例 9

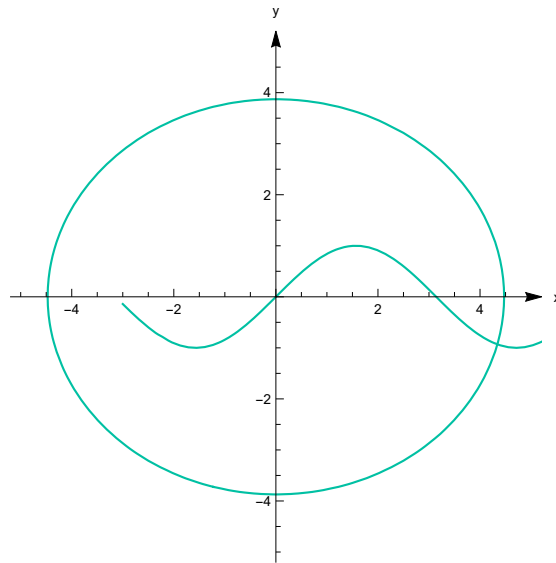


```

\begin{wolframGraphics}{wolframStroke}
fp1 = ContourPlot[
  x^2 + y^2 == 4, {x, -1.3, 0.6}, {y, -2.4, 3.2},
  AspectRatio->(2.4+3.2)/(1.3+0.6), ContourStyle->Red
];
fp2 = ContourPlot[
  x^2 + y^2 == 4, {x, -3, 3}, {y, -3, 3},
  AspectRatio->1, ContourStyle->RGBColor["#00C0A3"],
  AxesOrigin->{0, 0}, Axes->True
];
fp3 = ContourPlot[
  {x^2 + y^2 == 4, x^2 + Sin[y] == 1},
  {x, -2.5, 2.5}, {y, -3, 3},
  ContourStyle->{
    {RGBColor["#00C0A3"], Thickness[0.01]},
    {RGBColor["#FF9671"], Thickness[0.01]}
  },
  AspectRatio->(3+3)/(2.5+2.5), AxesOrigin->{0,0},
  Axes->True, Frame->False,
  AxesStyle->Arrowheads[{0,0.01}]
]
FIGURE = Show[fp2, fp1, fp3];
\end{wolframGraphics}
\includegraphics[width=.5\linewidth]{\wolframOutputFile}

```

3.2 案例 10



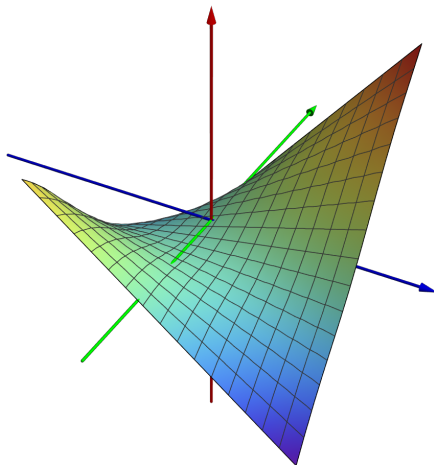
```

\begin{wolframGraphics}{wolfram2Dplot}
plotFunction[fun_, xlimits_, ylimits_] := ContourPlot[
  fun, xlimits, ylimits,
  ContourStyle->{
    RGBColor["#00C0A3"],
    Thickness[0.004]
  },
  AspectRatio->((xlimits[[2]]//Abs) + (xlimits[[3]]//Abs))
    /((ylimits[[2]]//Abs) + (ylimits[[3]]//Abs)),
  AxesOrigin->{0,0},
  Axes->True, Frame->False,
  AxesStyle->Arrowheads[{0, 0.03}],
  AxesLabel->{"x", "y"},
  PlotRange -> Full
]

xlimits = {x, -3, 6};
ylimits = {y, -4, 5};
fp1 = plotFunction[y==Sin[x], xlimits, ylimits];
fp2 = plotFunction[x^2/4 + y^2/3 == 5, {x, -5, 5}, {y, -5, 5}];
FIGURE = Show[fp2, fp1];
\end{wolframGraphics}
\includegraphics[width=.5\linewidth]{\wolframOutputFile}

```

3.3 案例 11

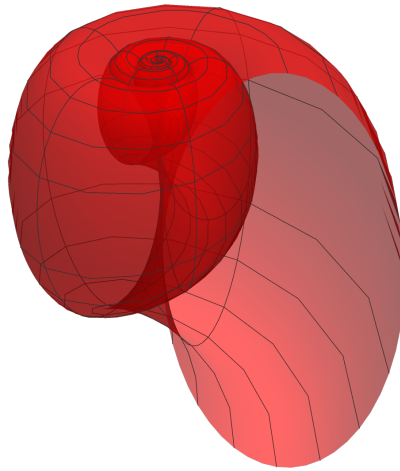


```

\begin{wolframGraphics}{wolfram3DAxis}
(* 1. 定义一个产生箭头的命令 *)
arrow[start_, end_, type_] := Graphics3D[
  { type,
    { Arrowheads[.02], Arrow[Tube[{start, end}, 0.06]]}
  }, Boxed->False
];
(* 2. 创建三个坐标轴的箭头, 使用颜色进行区分 *)
xaxis = arrow[{-10, 0, 0}, {10, 0, 0}, Blue];
yaxis = arrow[{0, -10, 0}, {0, 10, 0}, Green];
zaxis = arrow[{0, 0, -10}, {0, 0, 10}, Red];
(* 3. 展示在同一坐标轴 *)
axis = {xaxis, yaxis, zaxis};
(* 4. 绘制一个函数由于测试 *)
fp4 = Plot3D[
  0.4*x + 0.2*Sin[y] + 0.2*x*y,
  {x, -5, 7}, {y, -6, 4},
  ColorFunction->"Rainbow"
];
(* 5. 显示三维函数图像和坐标轴 *)
FIGURE = Show[axis, fp4]
\end{wolframGraphics}
\includegraphics[width=.5\linewidth]{\wolframOutputFile}

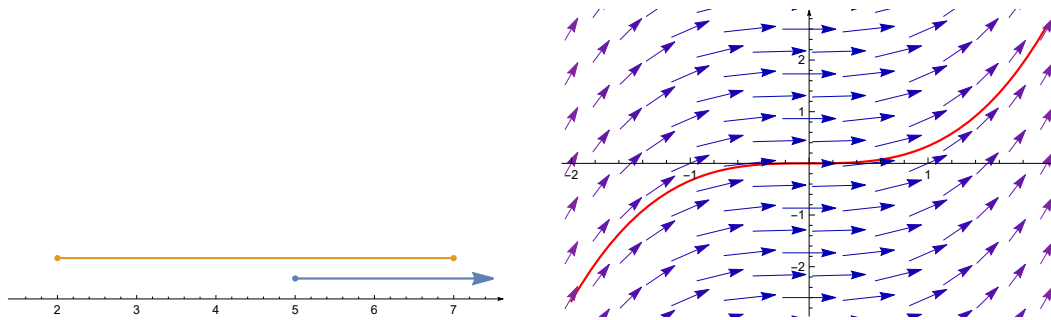
```

3.4 案例 12



```
\begin{wolframGraphics}{wolfram3DParametric}
FIGURE = ParametricPlot3D[
  {1.16^v*Cos[v]*(1+Cos[u]), -1.16^v*Sin[v]*(1+Cos[u]), -2 1.16^v*(1+Sin[u])},
  {u, 0, 2*Pi}, {v, -15, 6},
  PlotStyle->{Opacity[0.6],Red},
  PlotRange->All, PlotPoints->25,
  Axes->False, Boxed->False
];
\end{wolframGraphics}
\includegraphics[width=.4\linewidth]{\wolframOutputFile}
```

3.5 案例 13



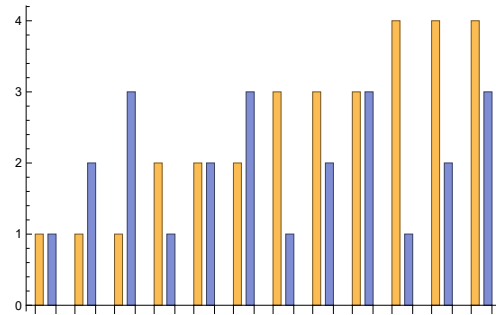
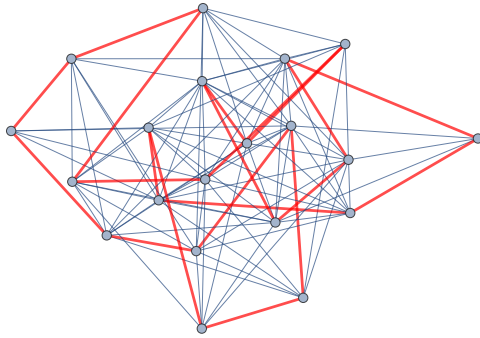
```

\begin{wolframGraphics}{wolframLine-I}
FIGURE = NumberLinePlot[
  { Interval[{5, Infinity}], Interval[{2, 7}] },
  AxesStyle->Arrowheads[{0, 0.01}]
];
\end{wolframGraphics}
\edef\mmaOutputTmp{\wolframOutputFile}

\begin{wolframGraphics}{wolframLine-II}
fvec = VectorPlot[
  {1, x^2}, {x, -4, 4}, {y, -4, 4},
  AxesOrigin->{0, 0}, Axes->False, Frame->False
];
fp = Plot[
  1/3*x^3, {x, -2, 2}, PlotStyle->Red,
  AxesStyle->Arrowheads[{0, 0.01}]
];
FIGURE = Show[fp, fvec];
\end{wolframGraphics}
\includegraphics[width=.45\linewidth]{\mmaOutputTmp}\quad
\includegraphics[width=.45\linewidth]{\wolframOutputFile}

```

3.6 案例 14



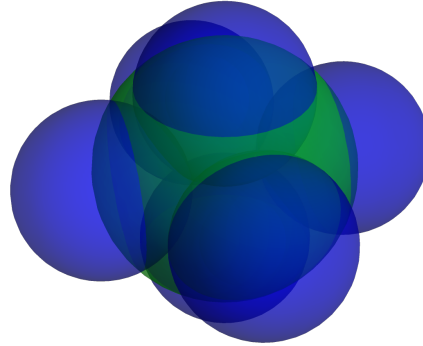
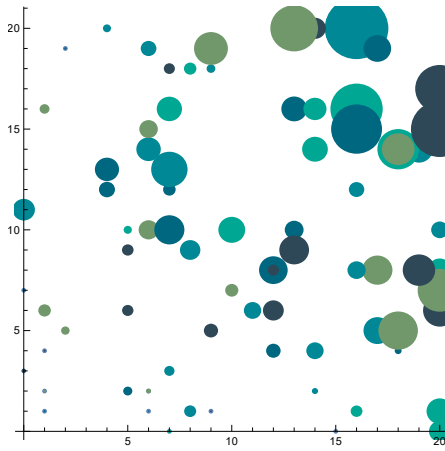
```

\begin{wolframGraphics}{wolframHamiltonian}
g = RandomGraph[{20, 100}];
h = FindHamiltonianCycle[g];
FIGURE = HighlightGraph[g, Style[h, Directive[Thick, Red]]];
\end{wolframGraphics}
\edef\mmaOutputTmp{\wolframOutputFile}

\begin{wolframGraphics}{wolframStatistic}
FIGURE = BarChart[Flatten[Table[{i, j}, {i, 1, 4}, {j, 1, 3}], 1]];
\end{wolframGraphics}
\includegraphics[width=.45\linewidth]{\mmaOutputTmp}\quad
\includegraphics[width=.45\linewidth]{\wolframOutputFile}

```

3.7 案例 15



```

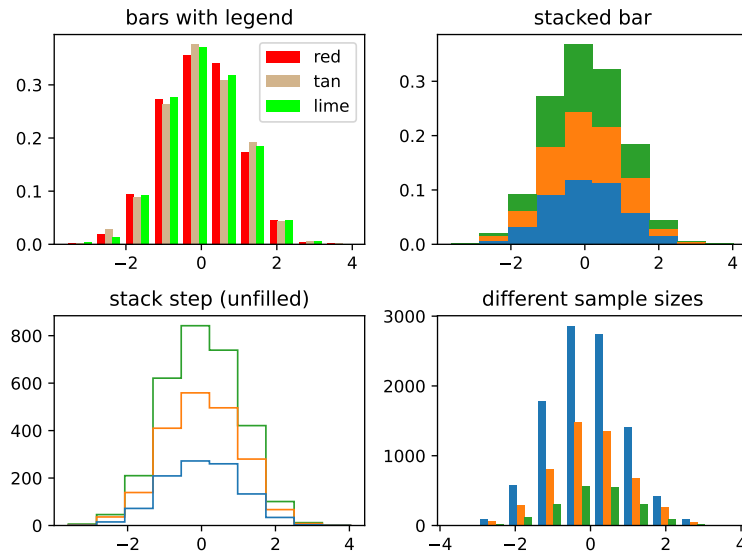
\begin{wolframGraphics}{wolfram2DBall1}
xls = RandomInteger[{0, 20}, 80];
yls = RandomInteger[{0, 20}, 80];
xycoor = {xls, yls} // Transpose;
color = { RGBColor["#00A894"], RGBColor["#008896"], RGBColor["#006780"],
RGBColor["#2F4858"], RGBColor["#70986B"]};
fp1 = Table[
  Graphics[{ color[[RandomInteger[{1, 5}]]],
    Disk[xycoor[[i]], RandomReal[{0, 0.05}]]*#1+RandomReal[{0,
0.05}]]*#2&[xycoor[[i]][[1]], xycoor[[i]][[2]]]
}], {i, 1, 80}
];
fp2 = ListPlot[xycoor, AspectRatio->(Max[yls])/(Max[xls])];
FIGURE = Show[fp2, fp1];
\end{wolframGraphics}
\edef\mmaOutputTmp{\wolframOutputFile}

\begin{wolframGraphics}{wolfram3DBall1}
FIGURE = Graphics3D[{
  Blue, Opacity[0.5], Sphere[{0.5, 0.5, 0}, 0.5],
  Blue, Opacity[0.5], Sphere[{-0.5, -0.5, 0}, 0.5],
  Blue, Opacity[0.5], Sphere[{0.5, -0.5, 0}, 0.5],
  Blue, Opacity[0.5], Sphere[{-0.5, 0.5, 0}, 0.5],
  Blue, Opacity[0.5], Sphere[{0, 0, 0.5}, 0.5],
  Blue, Opacity[0.5], Sphere[{0, 0, -0.5}, 0.5],
  Green, Sphere[{0, 0, 0}, 0.75]
}], Boxed->False
];
\end{wolframGraphics}
\includegraphics[width=.4\linewidth]{\mmaOutputTmp}\qqquad
\includegraphics[width=.4\linewidth]{\wolframOutputFile}

```

4 python 库

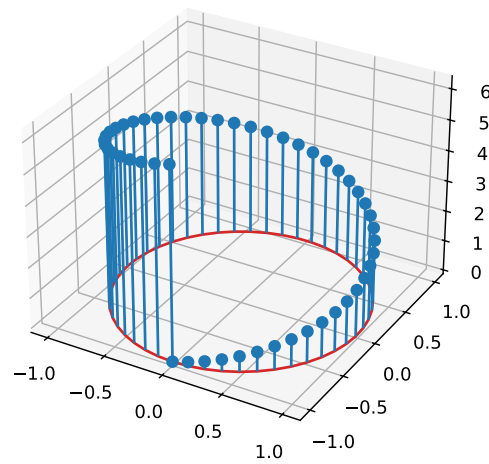
4.1 案例 16



```
\begin{pyfig}{pyfigExampleA}{pyfig-A.pdf}
# https://matplotlib.org/stable/gallery/lines_bars_and_markers/histogram_demo.html
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(19680801)
n_bins = 10
x = np.random.randn(1000, 3)
fig, ((ax0, ax1), (ax2, ax3)) = plt.subplots(nrows=2, ncols=2)
colors = ['red', 'tan', 'lime']
ax0.hist(x, n_bins, density=True, histtype='bar', color=colors, label=colors)
ax0.legend(prop={'size': 10})
ax0.set_title('bars with legend')
ax1.hist(x, n_bins, density=True, histtype='bar', stacked=True)
ax1.set_title('stacked bar')
ax2.hist(x, n_bins, histtype='step', stacked=True, fill=False)
ax2.set_title('stack step (unfilled)')
x_multi = [np.random.randn(n) for n in [10000, 5000, 2000]]
ax3.hist(x_multi, n_bins, histtype='bar')
ax3.set_title('different sample sizes')
fig.tight_layout()
\end{pyfig}
\includegraphics[width=.7\linewidth]{\pyfigOutputFile}
```

4.2 案例 17



```

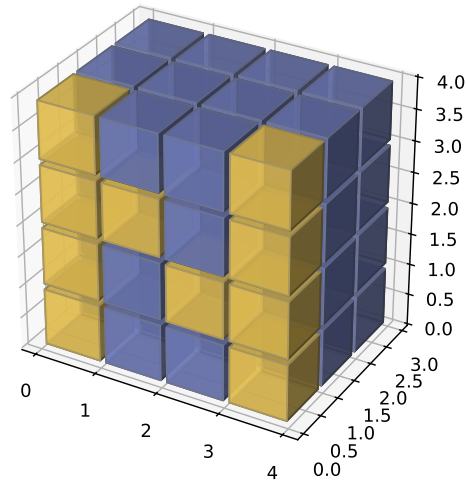
\begin{pyfig}{pyfigExampleB}{pyfig-B.pdf}
# https://matplotlib.org/stable/gallery/mplot3d/stem3d_demo.html
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import numpy as np

theta = np.linspace(0, 2*np.pi)
x = np.cos(theta - np.pi/2)
y = np.sin(theta - np.pi/2)
z = theta

fig, ax = plt.subplots(subplot_kw=dict(projection='3d'))
ax.stem(x, y, z)
\end{pyfig}
\includegraphics[width=.75\linewidth]{\pyfigOutputFile}

```

4.3 案例 18



```
\begin{pyfig}{pyfigExampleC}{pyfig-C.pdf}
# https://matplotlib.org/stable/gallery/mplot3d/voxels_numpy_logo.html
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import numpy as np

def explode(data):
    size = np.array(data.shape)*2
    data_e = np.zeros(size - 1, dtype=data.dtype)
    data_e[::2, ::2, ::2] = data
    return data_e

# build up the numpy logo
n_voxels = np.zeros((4, 3, 4), dtype=bool)
n_voxels[0, 0, :] = True
n_voxels[-1, 0, :] = True
n_voxels[1, 0, 2] = True
n_voxels[2, 0, 1] = True
facecolors = np.where(n_voxels, '#FFD65DC0', '#7A88CC0')
edgecolors = np.where(n_voxels, '#BFAB6E', '#7D84A6')
filled = np.ones(n_voxels.shape)

# upscale the above voxel image, leaving gaps
filled_2 = explode(filled)
fcolors_2 = explode(facecolors)
```

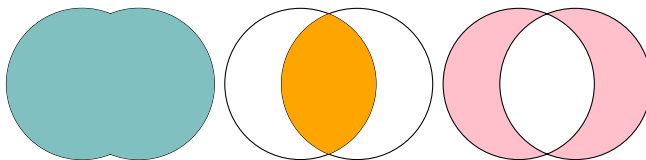
```
ecolors_2 = explode(edgecolors)

# Shrink the gaps
x, y, z = np.indices(np.array(filled_2.shape) + 1).astype(float) // 2
x[0::2, :, :] += 0.05
y[:, 0::2, :] += 0.05
z[:, :, 0::2] += 0.05
x[1::2, :, :] += 0.95
y[:, 1::2, :] += 0.95
z[:, :, 1::2] += 0.95

ax = plt.figure().add_subplot(projection='3d')
ax.voxels(x, y, z, filled_2, facecolors=fcolors_2, edgecolors=ecolors_2)
ax.set_aspect('equal')
\end{pyfig}
\includegraphics[width=.75\linewidth]{\pyfigOutputFile}
```

5 l3draw 库

5.1 案例 19

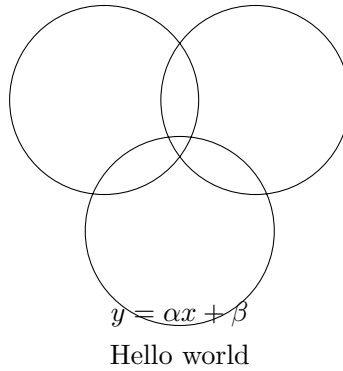


```
% union
\begin{Zdraw}
  \zxscale {0.5} \zyscale {0.5}
  \zcirc {2cm, 0}{2cm} \zcirc {3.5cm, 0}{2cm}
  \zusepath[draw, clip] \zfcolor {teal!50}
  \zrect {-10cm, -10cm}{10cm, 10cm}
  \zusepath[fill]
\end{Zdraw}

% intersection
\begin{Zdraw}
  \zxscale {0.5} \zyscale {0.5}
  \zcirc {3.5cm, 0}{2cm} \zusepath[draw]
  \zcirc {2cm, 0}{2cm} \zusepath[clip, draw]
  \zfcolor {orange} \zcirc {3.5cm, 0}{2cm}
  \zusepath[fill]
\end{Zdraw}

% difference
\begin{Zdraw}
  \zxscale {0.5} \zyscale {0.5}
  \zfevenodd \zfcolor {pink}
  \zcirc {2cm, 0}{2cm} \zcirc {3.5cm, 0}{2cm}
  \zusepath[draw, fill]
\end{Zdraw}
```

5.2 案例 20

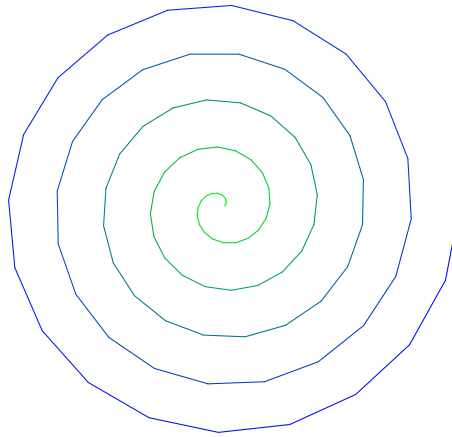


```

\begin{Zdraw}
  % draw circle
  \zxscale {.5} \zyscale {.5}
  \zcirc {-2cm, 0}{2.5cm}
  \zcirc {2cm, 0}{2.5cm}
  \zcirc {0, -2*sqrt(3)cm}{2.5cm}
  % add text
  \znewtext \text{a}
  \zsetvtext \text{a} {6em}{\$y=\alpha x + \beta\$\\ Hello~ world}
  \zscaletext \text{a} {2}{2}
  \zputtext \text{a} {hc}{b}{0, -7cm}
  \zusepath[draw]
\end{Zdraw}

```

5.3 案例 21



```

\ExplSyntaxOn
% Data Source: https://tex.stackexchange.com/a/721052/294585
\ztool_read_file_as_seq:neN
  {\c_false_bool}{gradient.data}
  \l_tmpa_seq % seq(without outer brace)={0, 0}, {0.03, 0.01}, ..., {3.14, 0}.
\cs_set:Npn \color_gradient:n #1
  { \color_select:n {blue!#1!green} }
\cs_generate_variant:Nn \color_gradient:n {e}

% Draw those segments
\draw_begin: \draw_cap_round:
\draw_xvec:n {1cm, 0}
\draw_yvec:n {0, 1cm}
\draw_path_moveto:n {\draw_point_vec:nn {0.785}{0}}
\int_step_inline:nnn {2}{\fp_eval:n {\seq_count:N \l_tmpa_seq-1}}
{
  \seq_set_split:Nne \l_tmpb_seq {,}{\seq_item:Nn \l_tmpa_seq {#1}}
  \seq_set_split:Nne \l_tmpc_seq {,}{\seq_item:Nn \l_tmpa_seq {\fp_eval:n {#1+1}}}
  \color_gradient:e {\fp_eval:n {#1*100/\seq_count:N \l_tmpa_seq}}
  \draw_path_moveto:n {
    \draw_point_vec:nn {\seq_item:Nn \l_tmpb_seq {1}}
    {\seq_item:Nn \l_tmpb_seq {2}}
  }
  \draw_path_lineto:n {
    \draw_point_vec:nn {\seq_item:Nn \l_tmpc_seq {1}}
    {\seq_item:Nn \l_tmpc_seq {2}}
  }
  \draw_path_use_clear:n {draw}
}
\draw_path_use_clear:n {draw} \draw_end:
\ExplSyntaxOff

```